

**Федеральное государственное бюджетное образовательное учреждение высшего
образования
«РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА и
ГОСУДАРСТВЕННОЙ СЛУЖБЫ
при Президенте Российской Федерации»**

КОЛЛЕДЖ МНОГОУРОВНЕВОГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ

Специальность 09.02.07 Информационные системы и программирование

УТВЕРЖДАЮ
Зам. директора КМПО РАНХиГС
_____ С.Ф. Гасанов
« _____ » _____ 2026 г.

ДИПЛОМНЫЙ ПРОЕКТ

**на тему: «Корпоративная система управления
выездным сервисом и ремонтом. Разработка модуля
аналитики и интеграций»**

Выполнил студент группы	_____	/Э.А. Шогенова/
Руководитель	_____	/Д.О. Никифоров/
Нормоконтролер	_____	/Д.О. Никифоров/
Консультант по технико-экономическому обоснованию проекта	_____	/О.М. Бутакова/
Старший консультант	_____	/Ф.В. Кусков/

**МОСКВА
2026 г.**

РЕФЕРАТ

КОРПОРАТИВНАЯ СИСТЕМА, АНАЛИТИКА ВЫЕЗДНОГО СЕРВИСА, SLA, KPI, ETL, БАЗА ДАННЫХ, PYTHON, SQLALCHEMY, MYSQL, PYQT6, ВИЗУАЛИЗАЦИЯ, ЭКОНОМИЧЕСКАЯ ЭФФЕКТИВНОСТЬ

Объект исследования – процесс управления и аналитики выездного сервисного обслуживания оборудования.

Цель работы – разработка программного комплекса для автоматизации сбора, обработки и визуализации данных о заявках, ремонтах, показателях эффективности инженеров и соблюдении уровней сервиса (SLA).

В процессе работы проведён анализ предметной области, выполнено сравнение существующих решений, обоснован выбор технологий. Разработана реляционная база данных (MySQL), реализован слой ORM (SQLAlchemy 2.x), созданы репозитории и сервисы бизнес-логики. Построены ETL-пайплайны для импорта данных из Bitrix24 (mock), 1С (mock) и Excel. Реализованы модули расчёта KPI и SLA, прогнозирования спроса скользящим средним, генерации аналитических отчётов в форматах Excel и PDF. Разработан графический интерфейс на PyQt6 с шестью экранами: дашборд, заявки, ремонты, аналитика, интеграции, отчёты. Проведено тестирование (unit-тесты на примере SLA-калькулятора). Выполнена оценка экономической эффективности внедрения: срок окупаемости 3 месяца, чистый экономический эффект 4,47 млн руб./год.

Результаты работы могут быть использованы в сервисных организациях для повышения управляемости и оптимизации затрат.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	10
1.1 Процессы выездного сервиса и ремонта.....	10
1.2 Обзор и сравнительный анализ существующих решений.....	15
1.3 Обоснование выбора технологического стека.....	18
1.4 Нормативно-правовое регулирование разработки программного продукта в Российской Федерации.....	21
2 ПРОЕКТИРОВАНИЕ, РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ.....	24
2.1 Архитектура программного комплекса.....	24
2.2 Проектирование базы данных.....	30
2.3 Реализация backend-логики системы.....	35
2.3.1 Реализация репозиторий.....	36
2.3.2 Реализация сервисного слоя.....	38
2.3.3 Управление транзакциями.....	41
2.4 Аналитические алгоритмы и модули.....	43
2.4.1 Расчет SLA.....	44
2.4.2 KPI инженеров.....	46
2.4.3 Прогнозирование спроса.....	47
2.4.4 Дескриптивная статистика ремонтов.....	48
2.5 Пользовательский интерфейс.....	49
2.5.1 Страница «Дашборд».....	51
2.5.2 Страница «Заявки».....	52
2.5.3 Страница «Ремонты».....	53
2.5.4 Страница «Аналитика».....	54
2.5.5 Страница «Интеграции».....	55
2.5.6 Страница «Отчёты».....	57

2.5.7	<u>Стилизация интерфейса.....</u>	<u>58</u>
2.6	<u>Интеграция с внешними сервисами.....</u>	<u>59</u>
2.6.1	<u>Интеграция с Bitrix24.....</u>	<u>60</u>
2.6.2	<u>Интеграция с 1С.....</u>	<u>61</u>
2.6.3	<u>Импорт данных в Excel.....</u>	<u>62</u>
2.6.4	<u>Логирование и аудит интеграций.....</u>	<u>63</u>
2.7	<u>Обеспечение информационной безопасности.....</u>	<u>63</u>
2.8	<u>Тестирование.....</u>	<u>65</u>
3	<u>ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ.....</u>	<u>68</u>
3.1	<u>Расчёт затрат на разработку.....</u>	<u>69</u>
3.2	<u>Оценка экономического эффекта от внедрения.....</u>	<u>71</u>
3.3	<u>Оценка рисков проекта.....</u>	<u>74</u>
3.4	<u>Сравнение разработки собственного продукта с покупкой готового решения.....</u>	<u>76</u>
	<u>ЗАКЛЮЧЕНИЕ.....</u>	<u>78</u>
	<u>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....</u>	<u>81</u>
	<u>ПРИЛОЖЕНИЕ А.....</u>	<u>83</u>
	<u>ПРИЛОЖЕНИЕ Б.....</u>	<u>85</u>

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API - Application Programming Interface

CRM - Customer Relationship Management

CRUD - Create, Read, Update, Delete

DTO - Data Transfer Object

ERP - Enterprise Resource Planning

ETL - Extract, Transform, Load

HTTP - HyperText Transfer Protocol

IT - Information Technology

JSON - JavaScript Object Notation

GUI - Graphical User Interface

KPI - Key Performance Indicator

ORM - Object-Relational Mapping

PDF - Portable Document Format

QSS - Qt Style Sheets

REST - Representational State Transfer

SAP - Systems, Applications and Products in Data Processing

SLA - Service Level Agreement

SQL - Structured Query Language

TCP/IP - Transmission Control Protocol / Internet Protocol

БД - База данных

СУБД - Система управления базами данных

ЭВМ - Электронно-вычислительная машина

ВВЕДЕНИЕ

Автоматизация сбора и анализа данных в компаниях, оказывающих услуги выездного ремонта и технического обслуживания оборудования, в современных условиях становится одним из ключевых факторов эффективного управления. Рост конкуренции на рынке сервисных услуг, повышение требований клиентов к скорости обработки заявок и качеству выполнения работ, а также необходимость контроля деятельности выездных инженеров требуют принятия управленческих решений на основе объективных данных. При отсутствии централизованной системы аналитики руководство вынуждено опираться на разрозненную информацию и субъективные оценки, что приводит к снижению качества планирования, увеличению издержек и ухудшению показателей обслуживания.

Для оценки эффективности сервисной деятельности необходимо учитывать широкий набор показателей: количество обработанных заявок, среднее время реакции и устранения неисправностей, процент успешно завершённых ремонтов, уровень соблюдения SLA, загрузку инженеров и динамику повторных обращений. На практике данные, необходимые для расчёта таких метрик, зачастую распределены между несколькими информационными системами. CRM-системы, например Bitrix24, используются для учёта клиентов и заявок, системы класса 1С:Предприятие - для ведения финансового и складского учёта, а отчётность нередко формируется вручную средствами электронных таблиц. Подобная фрагментация данных приводит к их дублированию, несогласованности и значительным временным затратам на подготовку аналитической информации.

Существующие корпоративные решения, предназначенные для автоматизации сервисного обслуживания, обладают широкими функциональными возможностями, однако их внедрение для предприятий среднего масштаба нередко оказывается экономически и организационно неоправданным. Такие платформы, как SAP Field Service Management и

Microsoft Dynamics 365 Field Service, ориентированы преимущественно на крупные компании и требуют существенных затрат на лицензирование, настройку и сопровождение. Кроме того, подобные системы часто содержат избыточный функционал, не востребованный в рамках конкретного предприятия. В связи с этим возрастает актуальность разработки специализированного решения, адаптированного к особенностям процессов выездного сервиса и обеспечивающего необходимый уровень аналитики без чрезмерных инфраструктурных затрат.

Объектом исследования являются бизнес-процессы выездного сервисного обслуживания оборудования, включающие этапы приёма заявки, распределения работ между инженерами, выполнения ремонта и закрытия обращения, а также сопровождающие их информационные потоки.

Предметом исследования выступают методы и средства автоматизации аналитической обработки данных сервисного обслуживания, включая интеграцию с внешними источниками данных, расчёт ключевых показателей эффективности, контроль выполнения SLA и формирование управленческой отчётности.

Целью дипломного проекта является разработка корпоративной системы аналитики выездного сервиса и ремонта, обеспечивающей централизованный сбор данных, автоматизированный расчёт ключевых показателей деятельности и предоставление инструментов поддержки управленческих решений.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) провести анализ предметной области, определить основные роли пользователей системы, этапы жизненного цикла сервисной заявки, а также ключевые показатели эффективности и источники данных;
- 2) выполнить сравнительный анализ существующих решений в области автоматизации сервисного обслуживания, включая SAP Field Service

Management, Microsoft Dynamics 365 Field Service и GLPI, и обосновать целесообразность разработки собственной системы;

3) обосновать выбор технологического стека, включающего язык программирования Python, СУБД MySQL, ORM-библиотеку SQLAlchemy, фреймворк PyQt6, а также библиотеки pandas и matplotlib для аналитической обработки данных и визуализации;

4) разработать структуру реляционной базы данных, отражающую сущности «клиент», «инженер», «заявка», «ремонт», «запчасть», «KPI-срез», «журнал интеграций» и связи между ними;

5) реализовать серверную логику приложения с использованием паттернов Repository и Service, обеспечив транзакционную целостность операций;

6) разработать ETL-процессы для загрузки данных из внешних источников: Bitrix24, 1С:Предприятие и файлов Excel;

7) реализовать аналитические модули, обеспечивающие расчёт SLA с учётом приоритетов заявок, агрегацию KPI сервисных инженеров, анализ загрузки персонала, прогнозирование спроса методом скользящего среднего и формирование статистики ремонтов;

8) разработать графический интерфейс пользователя на основе PyQt6, включающий модули дашбордов, управления заявками и ремонтами, аналитики, интеграций и отчётности;

9) обеспечить возможность экспорта аналитических отчётов в форматы Excel и PDF с автоматическим построением графиков;

10) выполнить тестирование ключевых компонентов системы, включая модуль расчёта SLA, и оценить экономическую эффективность внедрения разработанного решения.

Практическая значимость работы заключается в возможности адаптации разработанной системы для предприятий, осуществляющих сервисное обслуживание различного оборудования, включая компьютерную технику, бытовые устройства и промышленное оборудование. Использование единой

аналитической платформы позволяет сократить объём ручной обработки данных, повысить прозрачность контроля сервисных процессов и обеспечить более обоснованное принятие управленческих решений. Проведённая экономическая оценка показывает, что внедрение системы способствует снижению административных затрат и повышению производительности сервисного подразделения, что обеспечивает окупаемость проекта в течение нескольких месяцев эксплуатации.

Структура работы обусловлена поставленной целью и логикой исследования. В первой главе рассматриваются теоретические аспекты автоматизации сервисного обслуживания и анализируются существующие подходы к построению аналитических систем. Во второй главе представлены проектирование и разработка программного решения, включая архитектуру системы, структуру базы данных, механизмы интеграции, аналитические модули, пользовательский интерфейс и вопросы тестирования. Третья глава посвящена оценке экономической эффективности внедрения разработанной системы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Процессы выездного сервиса и ремонта

Для разработки эффективной системы аналитики необходимо учитывать специфику бизнес-процессов организаций, осуществляющих выездное сервисное обслуживание оборудования. В отличие от стационарного сервиса, деятельность выездных подразделений характеризуется территориальной распределённостью заявок, зависимостью сроков выполнения работ от транспортной логистики, ограниченными возможностями оперативного контроля персонала и высокой значимостью скорости реагирования на обращения клиентов. В таких условиях качество управления напрямую зависит от полноты и актуальности информации о ходе выполнения сервисных работ.

Анализ практики функционирования сервисных организаций позволяет выделить типовой жизненный цикл сервисной заявки, отражающий последовательность основных этапов обработки обращения клиента.

На первом этапе осуществляется поступление и регистрация заявки. Клиент обращается в сервисную организацию посредством телефонной связи, электронной почты либо веб-интерфейса. В информационной системе фиксируются сведения о заказчике, описание неисправности, предполагаемые сроки выполнения работ и уровень приоритета заявки. В зависимости от особенностей инфраструктуры предприятия создание заявок может осуществляться как вручную, так и автоматически — посредством интеграции с CRM-системами, например Bitrix24, либо путём импорта данных из внешних файлов.

После регистрации выполняется этап классификации и планирования работ. Ответственный сотрудник определяет тип оборудования, оценивает предполагаемую сложность ремонта и назначает инженера с учётом его квалификации, специализации и текущей загрузки. Одновременно для заявки устанавливаются нормативные сроки исполнения в соответствии с соглашением об уровне сервиса (SLA). Корректность планирования на данном

этапе оказывает существенное влияние на соблюдение сроков выполнения работ и равномерность распределения нагрузки между сотрудниками.

Следующим шагом становится назначение инженера и организация его выезда на объект. После распределения заявки конкретному исполнителю инженер получает уведомление о необходимости выполнения работ. С момента закрепления специалиста за заявкой система автоматически начинает отсчёт времени реакции на поступившее обращение. Это позволяет в дальнейшем проводить анализ соблюдения установленных SLA-показателей и оценивать своевременность обработки заявок. Параллельно заявке присваивается статус «назначена». Для оповещения участников процесса используется механизм уведомлений, реализованный посредством электронной почты и мобильного приложения.

На этапе выполнения работ инженер осуществляет диагностику, техническое обслуживание либо ремонт оборудования непосредственно на территории заказчика. В информационной системе фиксируются ключевые параметры проведённых работ, включая тип и модель устройства, описание выявленной неисправности, дату начала обслуживания, длительность ремонтных работ, перечень использованных запасных частей, а также стоимость предоставленных услуг. По мере выполнения работ система обновляет статусы заявки и ремонта, обеспечивая тем самым актуальное отображение текущего состояния обслуживания. После завершения работ выполняется фиксация итогов. Инженер указывает результат выполнения заявки, например успешное завершение, невозможность проведения ремонта либо отмену заявки с указанием причины. На основании этих данных система автоматически рассчитывает фактическое время выполнения заявки и сопоставляет его с установленными нормативами SLA.

При превышении допустимых значений нарушение регистрируется в системе для анализа. В случае использования запчастей информация об их списании передаётся в учётные подсистемы.

Заключительным этапом жизненного цикла является аналитическая обработка накопленных данных. Руководитель сервисного подразделения получает доступ к агрегированным показателям деятельности: количеству обработанных заявок, средней продолжительности ремонтов, уровню соблюдения SLA, эффективности инженеров и финансовым результатам сервисных операций. Сформированные отчёты могут экспортироваться в форматы Excel и PDF для последующего использования в управленческой деятельности.

В рамках рассматриваемых процессов можно выделить несколько основных ролей пользователей системы.

Диспетчер или администратор осуществляет регистрацию и распределение заявок, контролирует их текущее состояние и при необходимости выполняет переназначение исполнителей. Инженер отвечает за выполнение выездных работ и фиксацию результатов обслуживания. Руководитель сервисной службы использует аналитические данные для оценки эффективности подразделения и принятия управленческих решений. Клиент выступает заказчиком услуг и может получать уведомления о ходе выполнения заявки. Системный администратор отвечает за сопровождение инфраструктуры, настройку интеграционных механизмов, администрирование базы данных и организацию резервного копирования информации.

Для анализа эффективности работы сервисного подразделения используется система ключевых показателей эффективности (KPI). Одним из базовых критериев является количество заявок, обработанных инженером за определённый период. Этот показатель позволяет оценить уровень загруженности специалистов и их производительность.

Дополнительно анализируется доля успешно завершённых ремонтов, которая рассчитывается как отношение числа ремонтов, выполненных без повторных обращений, к общему количеству. Данный показатель отражает

качество предоставляемого сервиса и эффективность устранения неисправностей.

Отдельное внимание уделяется средней продолжительности выполнения ремонта. Этот параметр характеризует фактические временные затраты на восстановление работоспособности оборудования и применяется для оценки оперативности сервисного обслуживания.

В разработанной системе также используется интегральный показатель нагрузки инженера — workload score. Он рассчитывается по следующей формуле:

$$\text{workload_score} = \text{orders} + 1.15 * \text{repairs}$$

где:

- 1) orders — количество обработанных заявок,
- 2) repairs — количество завершённых ремонтов,
- 3) коэффициент 1,15 отражает повышенную трудоёмкость ремонтных работ по сравнению с обычной обработкой заявки.

Применение данного показателя позволяет более точно оценивать реальную нагрузку сотрудников и выявлять дисбаланс в распределении задач внутри сервисного подразделения. В отличие от простого подсчёта количества заявок, интегральный подход учитывает различия в сложности выполняемых операций, что повышает объективность оценки.

Важную роль в организации сервисного обслуживания играет контроль соблюдения соглашений об уровне сервиса (SLA). В системе для различных категорий приоритетов установлены нормативы времени выполнения заявок:

- 1) критический приоритет — не более 8 часов,
- 2) высокий приоритет — до 24 часов,
- 3) средний приоритет — до 48 часов,
- 4) низкий приоритет — до 96 часов.

Нарушение SLA фиксируется в случаях, когда фактическое время от регистрации до закрытия заявки превышает установленный норматив. Для заявок, находящихся в процессе выполнения, расчёт может осуществляться

относительно текущего момента времени, что позволяет заранее выявлять риск нарушения соглашений.

Следует отметить, что информационные потоки в сервисных организациях часто остаются разрозненными. Данные о клиентах и заявках обычно хранятся в CRM-системах, например Bitrix24, финансовая и складская информация — в системе 1С:Предприятие, а оперативные диспетчерские данные могут фиксироваться в электронных таблицах или даже на бумажных носителях. Такая фрагментация затрудняет формирование целостной аналитической картины и усложняет контроль бизнес-процессов.

Разрабатываемая система устраняет указанные недостатки за счёт внедрения механизмов ETL-загрузки данных из различных источников и их приведения к единой модели хранения. Это обеспечивает централизованный доступ к информации, снижает объём ручной обработки данных и формирует основу для автоматизированного анализа деятельности сервисного подразделения, что более подробно рассматривается во второй главе работы.

1.2 Обзор и сравнительный анализ существующих решений

На современном рынке представлены как коммерческие корпоративные платформы для управления выездным сервисным обслуживанием, так и решения с открытым исходным кодом, допускающие адаптацию под потребности конкретной организации. Анализ существующих систем необходим для определения их применимости в условиях среднего сервисного предприятия, а также для обоснования целесообразности разработки собственного программного решения.

В рамках исследования рассмотрены три характерных представителя данного класса систем: SAP Field Service Management как пример крупной enterprise-платформы, Microsoft Dynamics 365 Field Service как экосистемное решение, интегрированное с продуктами Microsoft, и GLPI как свободно распространяемая система с открытым исходным кодом.

Система SAP Field Service Management обеспечивает комплексную автоматизацию процессов сервисного обслуживания: управление заявками, планирование маршрутов, распределение инженеров, учёт запасных частей и поддержку мобильных рабочих мест технических специалистов. Платформа интегрируется с ERP-системами SAP и поддерживает интеллектуальные механизмы прогнозирования времени выполнения работ. Вместе с тем использование данного решения требует значительных затрат на лицензирование, внедрение и сопровождение. Для предприятий среднего масштаба подобные расходы зачастую являются экономически необоснованными. Кроме того, функциональные возможности системы во многих случаях оказываются избыточными для организаций с относительно простой структурой сервисных процессов.

Платформа Microsoft Dynamics 365 Field Service ориентирована на работу в экосистеме Microsoft и тесно интегрирована с другими продуктами компании, включая Dynamics 365 Sales, Power Platform и Microsoft Teams. Система поддерживает автоматизированное распределение заявок, инструменты маршрутного планирования и средства аналитики. Однако её

внедрение связано с регулярными лицензионными платежами и необходимостью интеграции с существующей ИТ-инфраструктурой предприятия. Для организаций, уже использующих 1С:Предприятие и Bitrix24, переход на платформу Microsoft сопряжён с дополнительными затратами и рисками миграции данных.

Система GLPI представляет собой бесплатное решение с открытым исходным кодом, предназначенное для управления ИТ-активами и обработки обращений пользователей. Преимуществами платформы являются отсутствие лицензионных платежей, возможность модификации исходного кода и наличие активного сообщества разработчиков. Функциональность системы ограничена задачами Help Desk и учёта оборудования. Для использования в качестве платформы управления выездным сервисом требуется доработка, включая реализацию аналитических модулей, механизмов расчёта SLA и интеграции с внешними системами. Трудоёмкость адаптации может оказаться сопоставимой с разработкой собственного программного решения.

Таблица 1 – Сравнение существующих решений для управления выездным сервисом

Критерий	SAP Field Service Management	Microsoft Dynamics 365 Field Service	GLPI	Собственная разработка
Начальная стоимость внедрения	Более 5 млн руб.	Более 3 млн руб.	Отсутствует, кроме затрат на доработку	1,53 млн руб.
Ежегодные лицензионные платежи	Значительные	От 150 долларов США в месяц на пользователя	Отсутствуют	Отсутствуют
Интеграция с 1С	Через платные адаптеры	Сложная интеграция	Не предусмотрена	Реализуется через собственный ETL-модуль
Интеграция с Bitrix24	Требует дополнительной разработки	Не предусмотрена	Не предусмотрена	Реализована
Поддержка мобильного приложения	Предусмотрена за дополнительную плату	Предусмотрена	Отсутствует	Разрабатывается в рамках проекта

Продолжение таблицы 1

Аналитика KPI и SLA	Расширенная	Расширенная	Ограниченная	Полностью адаптируемая
Возможность адаптации под специфику предприятия	Ограничена конфигурацией	Ограничена	Высокая	Полная
Зависимость от поставщика	Полная	Полная	Отсутствует	Отсутствует

Проведённый анализ показывает, что коммерческие корпоративные платформы ориентированы преимущественно на крупные организации и характеризуются высокой стоимостью владения. В свою очередь, решения с открытым исходным кодом требуют значительной адаптации и не обеспечивают необходимого уровня аналитической функциональности без существенной переработки. В этих условиях разработка собственной системы представляется наиболее рациональным вариантом для сервисного предприятия среднего масштаба. Такой подход позволяет учесть специфику бизнес-процессов организации, отказаться от постоянных лицензионных платежей и обеспечить полный контроль над развитием функциональности системы.

1.3 Обоснование выбора технологического стека

Разрабатываемая система предназначена для эксплуатации в условиях сервисного предприятия и ориентирована на использование локального сервера баз данных и рабочих станций под управлением операционных систем Windows или Linux. Выбор технологического стека осуществлялся с учётом требований к производительности, надёжности, масштабируемости, удобству сопровождения и наличию инструментов аналитической обработки данных.

В качестве основного языка программирования выбран Python. Данное решение обусловлено несколькими факторами. Во-первых, язык обладает развитой экосистемой библиотек для работы с базами данных, аналитики, визуализации и построения графических интерфейсов. Во-вторых, Python обеспечивает высокую скорость разработки благодаря лаконичному синтаксису и широкому набору готовых компонентов. Дополнительным преимуществом является кроссплатформенность, позволяющая использовать приложение без существенных изменений как в среде Windows, так и Linux.

Для хранения данных выбрана система управления базами данных MySQL. Её использование обусловлено высокой распространённостью в корпоративной среде, наличием бесплатной редакции MySQL Community Edition и поддержкой транзакционной обработки данных. СУБД обеспечивает эффективную работу с индексами, поддерживает обработку временных данных и позволяет масштабировать систему при увеличении объёма заявок и аналитической информации.

В качестве ORM-инструмента используется SQLAlchemy 2.x в декларативном стиле. Применение ORM позволяет отделить бизнес-логику приложения от SQL-запросов, упростить сопровождение схемы базы данных и обеспечить объектное представление сущностей предметной области. Использование типизированных конструкций SQLAlchemy повышает читаемость кода и снижает вероятность ошибок при работе с данными.

Графический интерфейс реализуется на основе библиотеки PyQt6. Выбор данного фреймворка связан с его кроссплатформенностью, высокой

производительностью и развитым набором пользовательских компонентов. PyQt6 предоставляет средства для построения сложных интерфейсов, работы с таблицами и интеграции графических элементов. Дополнительным преимуществом является возможность встраивания графиков matplotlib непосредственно в интерфейс приложения.

Для аналитической обработки данных используются библиотеки pandas и matplotlib. Библиотека pandas обеспечивает удобные средства агрегации, фильтрации и группировки данных, а также поддержку экспорта отчётов в формат Excel. Библиотека matplotlib применяется для построения диаграмм и графиков, используемых в аналитических отчётах и пользовательских дашбордах.

Интеграция с внешними источниками данных реализуется посредством ETL-механизмов. Для взаимодействия с Bitrix24 и 1С:Предприятие используются mock-клиенты, позволяющие демонстрировать работу системы без подключения к реальным API. При необходимости архитектура допускает замену тестовых компонентов на реальные REST-интеграции без изменения основной бизнес-логики. Загрузка данных из Excel осуществляется средствами библиотеки pandas с последующей нормализацией структуры данных.

Важной частью архитектуры является применение шаблонов проектирования Repository и Service. Паттерн Repository инкапсулирует доступ к данным и централизует выполнение запросов к базе данных. Паттерн Service используется для реализации бизнес-логики: расчёта SLA, формирования KPI, назначения инженеров и генерации отчётов. Для организации процессов загрузки данных применяется ETL-подход, предусматривающий последовательное выполнение операций извлечения, преобразования и загрузки данных.

Выбранный технологический стек позволяет реализовать полнофункциональную корпоративную систему без использования дорогостоящих коммерческих компонентов. При этом архитектура

приложения сохраняет возможность дальнейшего масштабирования и функционального расширения.

1.4 Нормативно-правовое регулирование разработки программного продукта в Российской Федерации

Разработка информационной системы, предназначенной для обработки данных клиентов и хранения сведений о сервисных работах, осуществляется в рамках действующего законодательства Российской Федерации. Соблюдение нормативно-правовых требований необходимо как для обеспечения законности обработки информации, так и для минимизации правовых и финансовых рисков.

Правовой статус программного обеспечения определяется положениями части IV Гражданского кодекса Российской Федерации. В соответствии со статьями 1259-1262 программа для ЭВМ рассматривается как объект авторского права. Исключительные права на программный продукт, созданный в рамках выполнения служебных обязанностей, принадлежат работодателю, если иное не предусмотрено договором. Для дипломного проекта данное положение означает возможность передачи прав на разработанную систему организации-заказчику при её практическом внедрении.

Общие требования к информационным системам устанавливаются Федеральным законом № 149-ФЗ «Об информации, информационных технологиях и о защите информации». Закон определяет необходимость обеспечения конфиденциальности, целостности и доступности данных, а также защиты информации от несанкционированного доступа.

Ключевое значение для рассматриваемой системы имеет Федеральный закон № 152-ФЗ «О персональных данных», поскольку база данных содержит сведения о клиентах и сотрудниках организации, включая контактную информацию и адреса обслуживания. Закон обязывает оператора персональных данных обеспечивать законность обработки информации, ограничивать доступ к данным и принимать организационные и технические меры защиты.

В рамках разработанной системы реализованы базовые механизмы обеспечения безопасности: ведение журналов интеграций и ограничение доступа к приложению на уровне локальной инфраструктуры предприятия. Вместе с тем для промышленной эксплуатации потребуются внедрение дополнительных средств защиты, включая ролевую аутентификацию пользователей, шифрование хранимых данных и разграничение прав доступа.

Требования к защите персональных данных в информационных системах дополнительно регламентируются Постановлением Правительства РФ № 1119. Для большинства сервисных организаций рассматриваемая система относится к третьему уровню защищённости персональных данных, что предполагает необходимость регистрации событий безопасности, контроля доступа и резервного копирования информации.

Следует учитывать и положения Федерального закона № 98-ФЗ «О коммерческой тайне». Информация о клиентах, стоимости работ, используемых запасных частях и условиях договоров может относиться к сведениям, составляющим коммерческую тайну предприятия. Соответственно, доступ к таким данным должен быть ограничен в соответствии с внутренними регламентами организации.

Несоблюдение требований законодательства в области обработки персональных данных может повлечь административную ответственность, предусмотренную статьёй 13.11 Кодекса Российской Федерации об административных правонарушениях. Для юридических лиц размеры штрафов могут достигать нескольких сотен тысяч рублей, что делает обеспечение нормативного соответствия не только правовой, но и экономической необходимостью.

Таким образом, анализ предметной области, исследование существующих решений, обоснование технологического стека и рассмотрение нормативно-правовых аспектов формируют теоретическую основу для последующей разработки корпоративной системы аналитики

выездного сервиса, практическая реализация которой рассматривается в следующей главе.

2 ПРОЕКТИРОВАНИЕ, РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ

2.1 Архитектура программного комплекса

Разрабатываемая система «Корпоративная аналитика выездного сервиса и ремонта» реализована на основе клиент-серверной архитектуры с разделением приложения на независимые функциональные уровни. Подобный подход обеспечивает модульность, упрощает сопровождение системы и позволяет изолировать бизнес-логику от механизмов хранения данных и пользовательского интерфейса.

Архитектура программного комплекса включает следующие основные компоненты: уровень представления, слой бизнес-логики, слой доступа к данным, систему хранения данных, ETL-подсистему интеграции, аналитические модули.

На рисунке 1 представлена компонентная диаграмма системы.

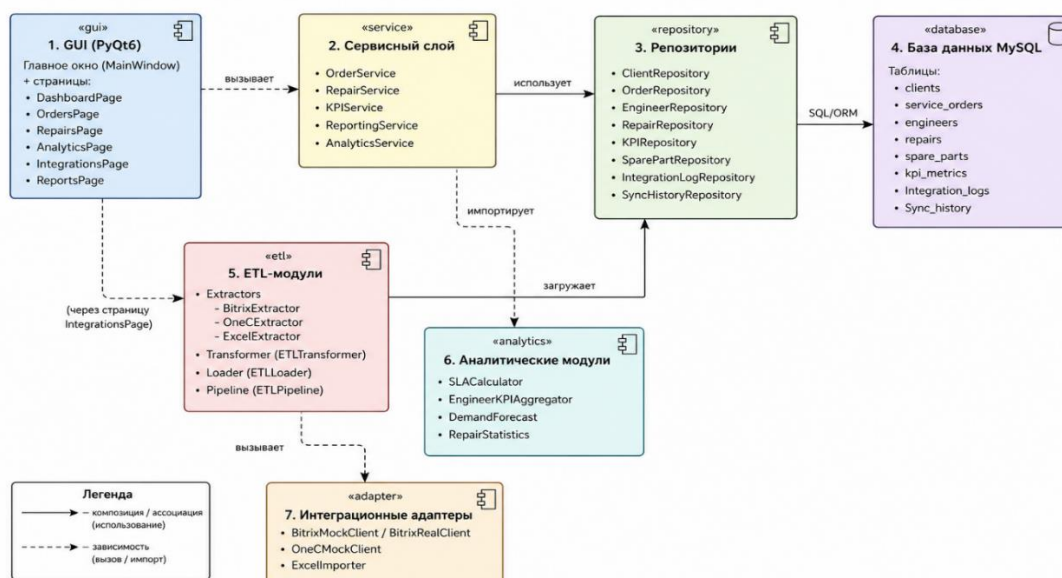


Рисунок 1 – Компонентная диаграмма системы

Верхний уровень архитектуры представлен графическим интерфейсом пользователя, реализованным на базе библиотеки PyQt6. Пользовательский интерфейс размещён в модуле ui и организован по принципу многостраничного приложения. Главное окно системы (MainWindow)

содержит боковую панель навигации и компонент `QStackedWidget`, обеспечивающий переключение между функциональными разделами приложения.

В структуре интерфейса выделены следующие страницы:

- а) `DashboardPage` – отображение сводной аналитики,
- б) `OrdersPage` – управление сервисными заявками;
- в) `RepairsPage` – работа с ремонтами и техническим обслуживанием,
- г) `AnalyticsPage` – просмотр аналитических показателей,
- д) `IntegrationsPage` – запуск и контроль процессов интеграции,
- е) `ReportsPage` – формирование и экспорт отчётов.

Каждая страница при инициализации получает доступ к сессии базы данных через контекстный менеджер `session_scope()`, который обеспечивает автоматическое управление транзакциями и корректное освобождение ресурсов. Такой подход позволяет централизовать обработку транзакций и снизить вероятность возникновения ошибок, связанных с управлением соединениями с базой данных.

Слой бизнес-логики реализован в пакете `services`. Его основное назначение заключается в обработке бизнес-правил и координации взаимодействия между пользовательским интерфейсом, репозиториями и аналитическими компонентами системы. Архитектура сервисного слоя построена на принципе разделения ответственности, что обеспечивает модульность и упрощает сопровождение кода.

В состав основных сервисов системы входят:

- 1) `OrderService` – управление жизненным циклом заявок, включая их создание, назначение исполнителей и изменение статусов;
- 2) `RepairService` – создание записей о ремонтах, фиксация результатов выполненных работ и учёт использованных запасных частей;
- 3) `KPIService` – расчёт и сохранение ключевых показателей эффективности деятельности;

- 4) ReportingService – формирование отчётной документации в форматах Excel и PDF;
- 5) AnalyticsService – агрегация и подготовка данных для дашбордов и аналитических представлений.

Выделение отдельного сервисного слоя позволяет исключить размещение бизнес-логики в пользовательском интерфейсе и обеспечивает возможность повторного использования функциональности в различных частях системы.

Доступ к данным реализован посредством репозиторий, размещённых в пакете repositories. Каждый репозиторий отвечает за работу с определённой сущностью предметной области и инкапсулирует SQL-запросы. В системе используются репозитории: ClientRepository, OrderRepository, EngineerRepository, RepairRepository, KPIRepository, SparePartRepository, IntegrationLogRepository, SyncHistoryRepository.

Репозитории построены с использованием возможностей SQLAlchemy 2.x и применяют современный декларативный стиль работы с запросами. Для выборки данных используются конструкции select(), а для оптимизации загрузки связанных сущностей — механизм joinedload. Применение репозиторий позволяет централизовать доступ к данным и повысить тестируемость приложения.

В качестве системы хранения данных используется СУБД MySQL. Структура базы данных описана в модуле models с использованием декларативного подхода SQLAlchemy. Все сущности наследуются от базового класса DeclarativeBase. Между таблицами определены связи через механизм relationship, а внешние ключи обеспечивают поддержание ссылочной целостности данных. Для зависимых сущностей предусмотрено каскадное удаление записей.

Интеграция с внешними источниками данных реализована посредством ETL-подсистемы, расположенной в пакете etl. Архитектура ETL-процессов основана на классической схеме Extract → Transform → Load.

На этапе извлечения данных используются специализированные компоненты, BitrixExtractor, OneCExtractor, ExcelExtractor.

Первые два компонента взаимодействуют с mock-клиентами, возвращающими тестовые JSON-данные. Архитектура предусматривает возможность перехода к реальным REST-запросам через настройку переменных окружения. обеспечивает импорт данных из файлов формата Excel с использованием библиотеки pandas.

Этап преобразования данных реализован посредством класса ETLTransformer, который выполняет нормализацию сведений, поступающих из различных источников. В процессе обработки статусы заявок и ремонтов приводятся к единому внутреннему формату, используемому системой. Так, например, значения in progress и processing преобразуются в единый статус in_progress. Помимо стандартизации данных, на данном этапе рассчитываются производные показатели, учитывая продолжительность ремонтных работ и признаки нарушения SLA.

Загрузка обработанных данных в базу происходит с помощью компонента ETLLoader. Добавление записей выполняется через необходимые репозитории, которые обеспечивают взаимодействие с уровнем хранения данных. При обнаружении отсутствующих связанных сущностей система создаёт необходимые записи, например сведения об инженерах, которые раньше не были зарегистрированы в бд.

Координация процессов синхронизации реализована с помощью сервисов BitrixSyncService, OneCSyncService и ExcelImportService. Указанные компоненты отвечают за запуск ETL-пайплайнов, контроль выполнения операций и ведение журналов синхронизации. Результаты обработки сохраняются в таблицах integration_logs и sync_history, что обеспечивает возможность мониторинга интеграционных процессов.

Отдельную подсистему составляют аналитические модули, размещённые в пакете analytics. Они реализованы в виде независимых классов

без хранения внутреннего состояния, что упрощает модульное тестирование и повторное использование компонентов.

В состав аналитической подсистемы входят:

- 1) SLACalculator – расчет соблюдения SLA;
- 2) EngineerKPIAggregator – агрегация KPI сервисных инженеров;
- 3) DemandForecast – прогнозирование нагрузки методом скользящего среднего;
- 4) RepairStatistics – вычисление статистических показателей ремонтов.

Аналитические модули принимают на вход специализированные dataclass-объекты и возвращают агрегированные структуры данных, используемые сервисным слоем и пользовательским интерфейсом.

Физическая структура размещения компонентов представлена на диаграмме развёртывания (рисунок 2).

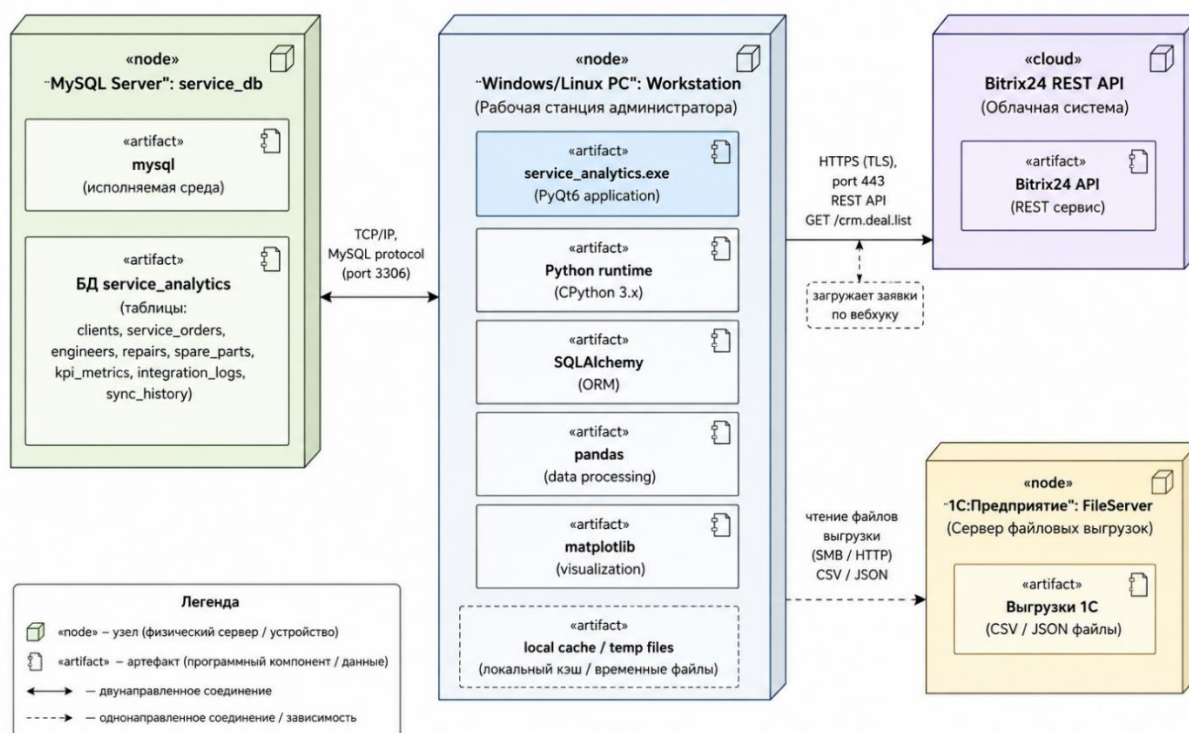


Рисунок 2 – Диаграмма развёртывания системы

Сервер базы данных может быть развёрнут как на отдельной машине внутри локальной сети предприятия, так и на рабочем сервере организации.

Клиентское приложение устанавливается на рабочие станции диспетчеров и администраторов и взаимодействует с MySQL по протоколу TCP/IP.

В текущей версии проекта интеграция с внешними системами реализована посредством mosk-компонентов и не требует постоянного доступа к внешней сети. При промышленной эксплуатации система может быть подключена к REST API Bitrix24 через веб-хуки, а также к файловым выгрузкам 1С:Предприятие посредством SMB-ресурсов или HTTP-доступа.

Таким образом, выбранная архитектура обеспечивает разделение ответственности между компонентами системы, упрощает сопровождение программного комплекса и создаёт основу для последующего масштабирования функциональности и интеграции с внешними сервисами.

2.2 Проектирование базы данных

Логическая модель базы данных разработана с учётом структуры бизнес-процессов сервисного обслуживания и отражает ключевые сущности предметной области: клиентов, инженеров, сервисные заявки, ремонты, запасные части, показатели эффективности и журналы интеграционных операций. Проектирование схемы выполнялось с целью обеспечения целостности данных, поддержки аналитических запросов и возможности дальнейшего масштабирования системы.

Физическая реализация базы данных выполнена на основе СУБД MySQL с использованием механизма хранения InnoDB. Выбор данного движка обусловлен поддержкой транзакций, внешних ключей и механизмов восстановления данных, что особенно важно для систем, работающих с финансовой и сервисной информацией.

ER-диаграмма базы данных представлена на рисунке 3.

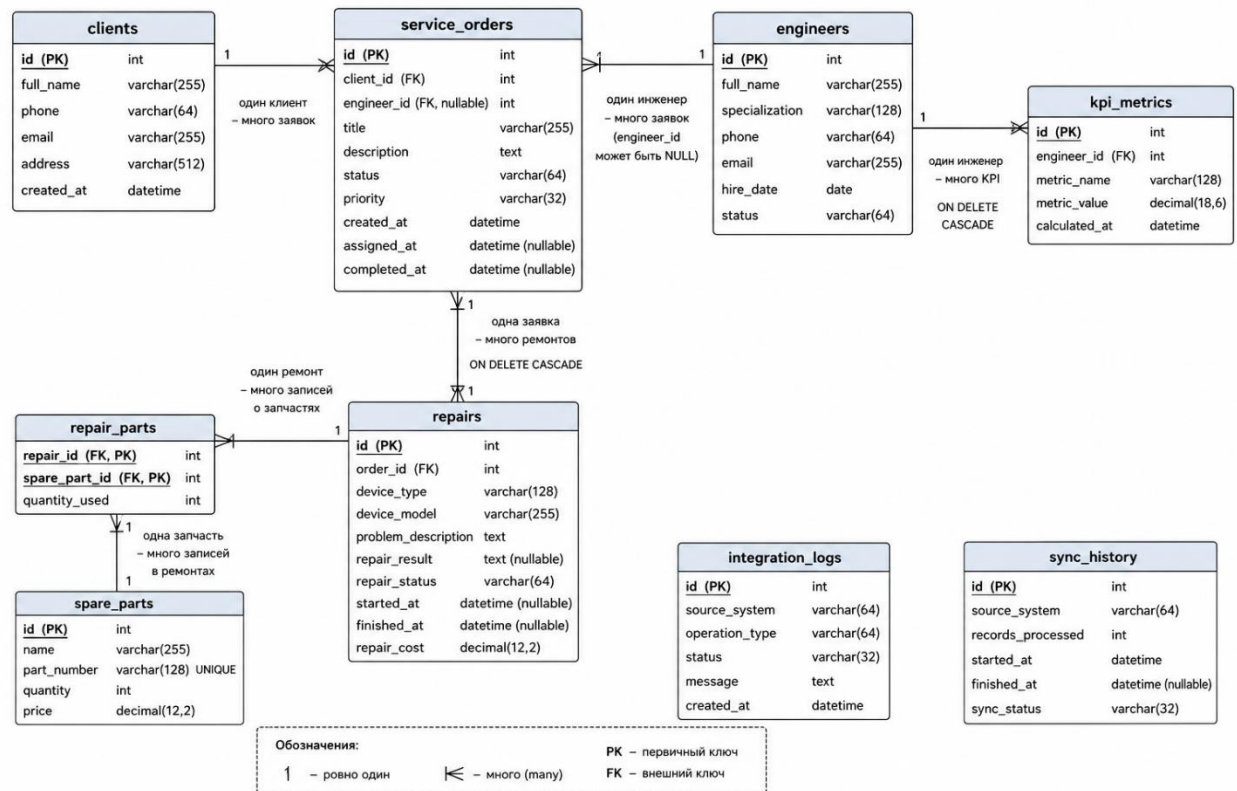


Рисунок 3 – ER-диаграмма базы данных

Структура таблиц реализована средствами ORM-библиотеки SQLAlchemy и размещена в отдельных модулях пакета models. Для каждой сущности определён собственный класс модели, описывающий поля таблицы, ограничения и связи между объектами предметной области.

В системе используются следующие основные таблицы.

Таблица 2 – Описание таблиц базы данных

Таблица	Назначение	Ключевые поля
clients	Хранение информации о клиентах, заказывающих сервисные услуги	id, full_name, phone, email, address, created_at
engineers	Хранение сведений о сервисных инженерах	id, full_name, specialization, phone, email, hire_date, status
service_orders	Хранение сервисных заявок и их состояния	id, client_id, engineer_id, title, description, status, priority, created_at, assigned_at, completed_at
repairs	Хранение информации о выполненных ремонтах	id, order_id, device_type, device_model, problem_description, repair_status, started_at, finished_at, repair_cost
spare_parts	Справочник запасных частей и складских остатков	id, name, part_number, quantity, price
repair_parts	Связь ремонтов и использованных запасных частей	repair_id, spare_part_id, quantity_used
kpi_metrics	Хранение рассчитанных КРІ-показателей инженеров	id, engineer_id, metric_name, metric_value, calculated_at
integration_logs	Журнал операций интеграции и ETL-процессов	id, source_system, operation_type, status, message, created_at
sync_history	История синхронизаций с внешними системами	id, source_system, records_processed, started_at, finished_at, sync_status

Центральной сущностью модели является таблица service_orders, содержащая информацию о сервисных заявках. Каждая заявка связана с клиентом посредством внешнего ключа client_id и при необходимости — с инженером через engineer_id. Такая структура позволяет хранить как распределённые, так и ещё не назначенные заявки.

Таблица repairs содержит сведения о ремонтных работах, выполняемых в рамках сервисной заявки. Между таблицами service_orders и repairs установлена связь типа «один ко многим», поскольку по одной заявке может

выполняться несколько ремонтов, например при повторных выездах или дополнительном обслуживании оборудования.

Для хранения информации о запасных частях в базе данных создана таблица `spare_parts`. Связь между ремонтами и используемыми деталями реализована через ассоциативную таблицу `repair_parts`, которая обеспечивает отношение типа «многие ко многим». Дополнительное поле `quantity_used` предназначено для указания количества одинаковых деталей, применённых в рамках конкретного ремонта. Такое решение соответствует принципам нормализации данных и позволяет исключить избыточное дублирование информации.

Результаты расчёта ключевых показателей эффективности инженеров сохраняются в таблице `kpi_metrics`. Накопление исторических значений КРІ обеспечивает возможность анализа динамики производительности сотрудников, выявления тенденций в их работе, а также формирования расширенной аналитической отчётности.

Для мониторинга процессов интеграции и синхронизации данных используются таблицы `integration_logs` и `sync_history`. В них фиксируются сведения об источниках данных, времени выполнения операций, количестве обработанных записей, а также статусах завершения процессов. Наличие журналов синхронизации позволяет выполнять аудит ETL-процессов и соответствует требованиям информационной безопасности, рассмотренным в подразделе 1.4.

При проектировании базы данных особое внимание уделялось вопросам производительности и обеспечению целостности данных. С целью ускорения обработки запросов были созданы индексы по наиболее часто используемым полям: `service_orders.status`, `service_orders.priority`, `service_orders.client_id`, `service_orders.engineer_id`, `repairs.repair_status`, `engineers.status`, `clients.email`, `integration_logs.source_system`, `sync_history.source_system`. Использование индексов сокращает время выполнения операций выборки при построении дашбордов, формировании отчётов и обработке данных.

Все поля, содержащие временные значения, реализованы с использованием типа `DateTime(timezone=True)`. Такой подход обеспечивает корректную обработку временных зон и позволяет избегать ошибок при расчёте SLA в распределённой инфраструктуре, когда сервер базы данных и клиентские рабочие станции могут находиться в различных часовых поясах.

Внешние ключи для зависимых сущностей настроены с использованием параметра `ondelete="CASCADE"`. Например, при удалении сервисной заявки автоматически удаляются связанные с ней записи о ремонтах. Аналогичный механизм применяется для удаления KPI-метрик инженеров. Для справочных таблиц каскадное удаление не используется, поскольку это может привести к потере критически важной информации о запчастях.

Для хранения денежных данных в таблице `repairs` используется тип `Numeric(12,2)`. Применение фиксированной точности позволяет избежать ошибок округления, которые характерны для типа `float`, что важно при расчёте стоимости ремонтных работ и при формировании финансовой отчётности.

Формирование структуры базы данных автоматизировано средствами SQLAlchemy. В модуле `database/schema.py` реализована функция `create_all_tables`, которая вызывает метод `Base.metadata.create_all`. Первичное заполнение бд тестовыми данными осуществляется с помощью скрипта `seed_data.py`.

Автоматизация создания схемы бд и её начального заполнения упрощает развёртывание демоверсии системы. Также, такой подход ускоряет процесс тестирования и снижает вероятность ошибок, которые могут возникнуть на этапе инициализации бд.

Таким образом, разработанная структура базы данных обеспечивает хранение и обработку информации о сервисных процессах, поддерживает аналитические сценарии использования и создаёт основу для надёжного функционирования корпоративной системы аналитики выездного сервиса и ремонта.

2.3 Реализация backend-логики системы

Серверная часть программного комплекса реализована на языке Python с использованием архитектурного разделения на слой доступа к данным и слой бизнес-логики. В основе backend-архитектуры лежат шаблоны проектирования Repository и Service, позволяющие изолировать SQL-запросы от прикладной логики системы. Такой подход повышает читаемость кода, упрощает сопровождение и обеспечивает возможность модульного тестирования отдельных компонентов.

2.3.1 Реализация репозитория

Слой доступа к данным реализован в пакете `repositories`. Каждый репозиторий отвечает за работу с одной сущностью предметной области и предоставляет набор методов для выполнения CRUD-операций, а также специализированных выборки, используемых аналитическими и сервисными модулями.

В качестве примера можно рассмотреть репозиторий `OrderRepository`, реализованный в файле `repositories/order_repository.py`. Данный компонент инкапсулирует операции работы с сервисными заявками.

Основные методы репозитория включают:

- 1) `get_by_id(order_id)` – загрузка заявки по идентификатору вместе со связанными сущностями;
- 2) `list_recent(limit)` – получение списка последних созданных заявок;
- 3) `filter_orders(status, engineer_id, search)` – фильтрация заявок по набору параметров;
- 4) `count_by_engineer(engineer_id)` – подсчёт количества заявок конкретного инженера;
- 5) `orders_created_between(start, end)` – выборка заявок за указанный период;
- 6) `add(order)` – добавление новой заявки в базу данных.

При загрузке связанных объектов используется механизм `joinedload`, позволяющий избежать проблемы множественных SQL-запросов при получении клиента, инженера и связанных ремонтов. Для построения выборки применяется декларативный стиль `SQLAlchemy 2.x` с использованием функции `select()`.

Метод `filter_orders()` реализует динамическое построение SQL-запроса в зависимости от переданных параметров. Для текстового поиска используется оператор `ilike`, обеспечивающий регистронезависимую фильтрацию записей.

Все репозитории работают через объект `Session`, передаваемый в конструктор класса и сохраняемый в поле `self.session`. Такой подход

позволяет нескольким репозиториям участвовать в рамках одной транзакции, управляемой через контекстный менеджер `session_scope()`.

Аналогичным образом реализованы `ClientRepository`, `EngineerRepository`, `RepairRepository`, `SparePartRepository`, `KPIRepository`, `IntegrationLogRepository`, `SyncHistoryRepository`.

Следует отметить, что репозитории не содержат бизнес-логики. Их задачей является исключительно формирование запросов, загрузка данных и сохранение объектов. Это обеспечивает чёткое разделение ответственности между компонентами системы.

2.3.2 Реализация сервисного слоя

Бизнес-логика системы реализована в пакете `services`. Сервисы используют репозитории в качестве механизма доступа к данным и инкапсулируют прикладные правила обработки сервисных операций.

Одним из ключевых компонентов является `OrderService`, расположенный в файле `services/order_service.py`. Данный сервис отвечает за управление жизненным циклом сервисных заявок.

Метод `assign_engineer(order_id, engineer_id)` выполняет назначение инженера на заявку. Перед обновлением данных осуществляется проверка существования заявки и инженера, а также контроль статуса инженера. Назначение допускается только для сотрудников со статусом `active`. После успешной проверки система обновляет поле `engineer_id`, переводит заявку в состояние `assigned` и фиксирует время назначения в UTC.

Метод `mark_completed(order_id)` переводит заявку в статус `completed` и устанавливает значение поля `completed_at`. В рамках демонстрационной версии проекта реализована упрощённая логика завершения заявки без дополнительной проверки связанных ремонтов.

Метод `list_orders()` представляет собой фасад над методом `filter_orders()` репозитория и используется интерфейсными компонентами системы для получения отфильтрованных списков заявок.

Сервис `RepairService`, реализованный в файле `services/repair_service.py`, отвечает за управление ремонтами и запасными частями.

Метод `create_repair()` создаёт запись о ремонте, связанного с конкретной сервисной заявкой. Если заявка ещё не находится в статусе `in_progress`, её состояние автоматически обновляется.

Метод `attach_spare_parts(repair_id, part_numbers)` обеспечивает привязку запасных частей к ремонту через связь «многие ко многим». Для каждой детали выполняется проверка существования по артикулу (`part_number`), после чего формируется запись в ассоциативной таблице.

Особую роль в системе играет сервис KPIService, реализующий вычисление аналитических показателей.

Метод `compute_global_repair_stats()` агрегирует данные о ремонтах и вычисляет долю успешных ремонтов, среднюю стоимость ремонта и среднюю продолжительность работ.

Для выполнения расчётов используется аналитический модуль `RepairStatistics`.

Метод `compute_sla_summary()` загружает сервисные заявки, преобразует их в проекционные объекты `OrderSLAInput` и передаёт их модулю `SLACalculator`, который выполняет анализ соблюдения SLA.

Наиболее сложным компонентом является метод `snapshot_engineer_kpis()`. Его задача заключается в агрегации данных по инженерам и формировании снимка KPI-показателей на определённый момент времени. В процессе выполнения вычисляются количество обработанных заявок, процент успешных ремонтов и интегральный показатель загрузки `workload_score`.

Показатель загрузки рассчитывается по формуле:

$$\text{workload_score} = \text{orders} + 1.15 \times \text{repairs}$$

Полученные значения сохраняются в таблицу `kpi_metrics` вместе с временной меткой, что позволяет анализировать динамику эффективности сотрудников.

Сервис `ReportingService` отвечает за генерацию отчётности.

Метод `export_excel(output_path)` формирует Excel-отчёт с несколькими листами: статистика SLA, данные по ремонтам, распределение заявок по дням, загрузка инженеров, KPI-показатели.

Формирование Excel-файлов выполняется посредством компонента `ExcelAnalyticsReport`.

Метод `export_pdf(output_path)` создаёт PDF-документ, содержащий сводные показатели и графики аналитики. Для генерации PDF используется модуль `matplotlib.backends.backend_pdf.PdfPages`.

Для построения графиков применяются распределение заявок по дням, загрузка инженеров и статистика статусов ремонтов.

Сервис `AnalyticsService` выполняет функцию фасада для аналитических экранов и дашбордов пользовательского интерфейса.

В состав сервиса входят методы:

- а) `sla_dashboard()` – получение сводки по SLA;
- б) `repair_dashboard()` – статистика ремонтов;
- в) `orders_per_day_frame()` – формирование `DataFrame` по количеству заявок;
- г) `engineer_workload_frame()` – данные о загрузке инженеров;
- д) `repairs_outcome_frame()` – распределение ремонтов по статусам;
- е) `forecast_next_day_orders()` – прогноз количества заявок методом скользящего среднего;
- ж) `executive_summary()` – формирование агрегированной управленческой сводки.

Использование сервисного слоя позволило централизовать бизнес-логику приложения и исключить её дублирование в пользовательском интерфейсе и репозиториях.

2.3.3 Управление транзакциями

Для обеспечения целостности данных в системе реализован механизм управления транзакциями на основе паттерна Unit of Work. В качестве централизованного механизма работы с транзакциями используется контекстный менеджер `session_scope()`, расположенный в модуле `database/session.py`.

Контекстный менеджер инкапсулирует процесс создания SQLAlchemy-сессии, фиксации изменений и обработки ошибок:

```
@contextmanager
def session_scope() -> Generator[Session, None, None]:
    if SessionLocal is None:
        configure_session_factory()
    session = SessionLocal()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()
```

В случае успешного выполнения операций происходит фиксация транзакции посредством метода `commit()`. При возникновении исключений все внесённые изменения автоматически откатываются с помощью `rollback()`, после чего ошибка повторно передаётся в вызывающий код. Независимо от результата выполнения, сессия гарантированно закрывается.

Применение единого механизма управления транзакциями обеспечивает согласованность изменений в базе данных, предотвращает

частичное сохранение данных, автоматически освобождает соединения и упрощает реализацию логики сервисного слоя.

Все сервисы и компоненты пользовательского интерфейса взаимодействуют с базой данных исключительно через `session_scope()`, что обеспечивает единообразный подход к управлению транзакциями во всех подсистемах приложения.

Таким образом, backend-архитектура системы реализует чёткое разделение ответственности между слоями, поддерживает транзакционную целостность данных и формирует основу для дальнейшего расширения функциональности программного комплекса.

2.4 Аналитические алгоритмы и модули

Аналитическая подсистема системы реализована в пакете `analytics` и предназначена для расчёта ключевых показателей эффективности, оценки соблюдения SLA, анализа загрузки инженеров и формирования прогнозных оценок спроса. Архитектурно модули аналитики выделены в отдельный слой и реализованы в виде классов без внутреннего состояния (`stateless`). Такой подход обеспечивает детерминированность вычислений, упрощает модульное тестирование и позволяет повторно использовать алгоритмы в различных частях системы (UI, сервисах, отчётности).

В качестве входных данных аналитические модули используют проекционные структуры (`DTO/dataclass`), формируемые на основе данных репозитория. Результаты вычислений возвращаются в виде агрегированных объектов или словарей, пригодных для отображения в интерфейсе и экспорта в отчёты.

2.4.1 Расчет SLA

Модуль расчёта SLA реализован в классе SLACalculator (analytics/sla_calculator.py) и предназначен для оценки соблюдения нормативных сроков выполнения сервисных заявок.

Основным элементом механизма контроля SLA выступает функция `target_hours(priority)`, которая определяет допустимое время выполнения заявки в зависимости от уровня её приоритета. Значения задаются через константный словарь `SLA_HOURS_BY_PRIORITY`, в котором для каждой категории установлен соответствующий временной лимит:

- а) low – 96 часов,
- б) medium – 48 часов,
- в) high – 24 часа,
- г) critical – 8 часов.

В случае указания некорректного или неизвестного приоритета применяется значение по умолчанию `DEFAULT_SLA_HOURS = 48`. Такой подход повышает устойчивость алгоритма к неполным или несогласованным данным и позволяет избежать ошибок при обработке заявок.

Расчёт SLA для отдельного обращения выполняется методом `evaluate_order(row, now)`. В процессе обработки определяется фактическая длительность выполнения заявки. Для завершённых обращений вычисляется интервал между моментами создания (`created_at`) и закрытия (`completed_at`). Для активных заявок используется разница между временем регистрации и текущим моментом. Отменённые заявки исключаются из анализа, поскольку не участвуют в оценке соблюдения SLA.

Для формирования сводной статистики используется метод `summarise(rows)`, который агрегирует результаты обработки заявок и рассчитывает основные показатели: общее количество обращений, долю

заявок, выполненных в рамках SLA, процент нарушений, а также среднее время обработки завершённых заявок.

Особое внимание при реализации алгоритма уделено обработке временных данных. Для исключения ошибок, связанных с различием часовых поясов, все временные метки приводятся к единому стандарту UTC с помощью вспомогательной функции `_aware`. Это особенно важно для распределённых систем, где сервер базы данных и клиентские приложения могут работать в разных часовых поясах.

Корректность работы алгоритма подтверждена набором модульных тестов. Ниже приведён пример тестового сценария:

```
def test_sla_breach_when_overdue_open_ticket():
    calc = SLACalculator()
    created = datetime(2025, 1, 1, 8, 0,
tzinfo=timezone.utc)
    row = OrderSLAInput(
        order_id=2,
        priority="high",
        status="in_progress",
        created_at=created,
        completed_at=None
    )
    now = created + timedelta(hours=48)
    res = calc.evaluate_order(row, now=now)
    assert res.breached is True
```

Данный тест демонстрирует корректное определение нарушения SLA для незавершённой заявки в случае превышения установленного нормативного времени выполнения.

2.4.2 KPI инженеров

Расчёт показателей эффективности инженеров реализован в модуле EngineerKPIAggregator (analytics/engineer_kpi.py). Данный компонент отвечает за агрегирование фактических данных о заявках и ремонтах и формирование сводных метрик по каждому сотруднику.

Метод `orders_per_engineer(facts)` выполняет группировку заявок по идентификатору инженера и определяет количество обработанных заявок на каждого сотрудника. В качестве входных данных используется список структур `EngineerOrderFact`.

Метод `success_rates(facts)` анализирует результаты ремонтов на основе списка `EngineerRepairFact` и вычисляет:

- а) количество успешных ремонтов,
- б) количество неуспешных ремонтов,
- в) процент успешности выполнения работ,
- г) среднюю длительность ремонта.

Метод `workload_index(orders_map, repairs_map)` формирует интегральный показатель нагрузки инженера:

$$\text{workload_score} = \text{orders} + 1.15 * \text{repairs}$$

Коэффициент 1.15 выбран эмпирически и отражает более высокую трудоёмкость ремонтных работ по сравнению с обработкой стандартной заявки. Полученные значения используются для ранжирования инженеров по уровню загрузки.

Формирование входных данных для данного модуля выполняется в сервисе `KPIService`, где данные извлекаются из репозитория и преобразуются в структуры `EngineerOrderFact` и `EngineerRepairFact`.

2.4.3 Прогнозирование спроса

Прогнозирование нагрузки реализовано в модуле DemandForecast (analytics/forecast_module.py). Алгоритм ориентирован на краткосрочное прогнозирование и предназначен для использования в оперативной аналитике.

Основной метод `moving_average_next_day(series, window=7)` реализует метод скользящего среднего. На вход подаётся временной ряд `DailyOrderVolume`, содержащий количество заявок по дням. Результатом является среднее значение за последние `window` дней, которое интерпретируется как прогноз количества заявок на следующий день.

При недостаточном объёме данных используется доступное количество наблюдений, что позволяет сохранять работоспособность алгоритма на ранних этапах эксплуатации системы.

Дополнительно реализован метод `trend_slope_orders_per_day(series)`, вычисляющий линейный тренд методом наименьших квадратов. Данный показатель используется в визуализации для оценки направления изменения нагрузки (рост, снижение или стагнация), однако не применяется в расчёте прогнозных значений в основной логике системы.

Следует отметить, что выбранный подход к прогнозированию ориентирован на простоту и интерпретируемость, что соответствует требованиям дипломного проекта и позволяет избежать избыточной сложности модели.

2.4.4 Дескриптивная статистика ремонтов

Модуль RepairStatistics (analytics/repair_statistics.py) предназначен для расчёта базовых статистических характеристик ремонтных работ на основе входного набора структур RepairFact.

В рамках анализа формируются следующие показатели:

- 1) count — общее количество выполненных ремонтов;
- 2) success_pct — доля успешно завершённых ремонтов;
- 3) failure_pct — доля неуспешных ремонтов;
- 4) avg_cost — средняя стоимость ремонта (с преобразованием Decimal в float);
- 5) avg_duration_hours — средняя продолжительность ремонта (рассчитывается только для записей с заполненными временными метками начала и окончания работ).

Полученные значения используются в аналитических дашбордах и при формировании PDF-отчётов, обеспечивая руководству предприятия возможность оценки общей эффективности сервисных операций.

Таким образом, аналитическая подсистема объединяет набор независимых, слабо связанных модулей, каждый из которых отвечает за отдельный аспект анализа данных: SLA, KPI, прогнозирование и дескриптивную статистику. Такое разделение обеспечивает расширяемость системы и позволяет в дальнейшем заменять или усложнять отдельные алгоритмы без изменения остальной архитектуры программного комплекса.

2.5 Пользовательский интерфейс

Пользовательский интерфейс системы реализован с использованием фреймворка PyQt6 и ориентирован на работу сотрудников сервисной организации: диспетчеров, администраторов и руководителей подразделений. При проектировании пользовательского интерфейса основное внимание уделялось скорости доступа к данным, наглядности представления аналитических показателей и сокращению количества действий при выполнении стандартных операций.

Главное окно приложения (MainWindow, файл ui/main_window.py) реализовано по принципу одностраничного интерфейса с боковой навигационной панелью и центральной областью отображения контента. Размер окна зафиксирован и составляет 1280×820 пикселей, что обеспечивает корректное отображение таблиц, графиков и аналитических панелей на большинстве рабочих станций.

Навигационная панель имеет фиксированную ширину 240 пикселей и выполнена в тёмной цветовой гамме (#1f2937). Центральная часть интерфейса содержит компонент QStackedWidget, который обеспечивает переключение между функциональными разделами без необходимости открытия дополнительных окон (см. рисунок 4).

Переключение между страницами реализовано посредством шести навигационных кнопок, каждая из которых связана с отдельным модулем пользовательского интерфейса (ui.pages). При переключении страницы вызывается метод refresh(), если он реализован в соответствующем классе. Это обеспечивает автоматическое обновление данных и отображение актуального состояния системы.



Рисунок 4 – Главное окно приложения

2.5.1 Страница «Дашборд»

Страница дашборда (`ui/pages/dashboard_page.py`) предназначена для оперативного мониторинга состояния сервисной службы и отображения ключевых показателей эффективности.

В верхней части интерфейса расположены четыре информационные карточки (KPI Cards), они отображают:

- а) процент соблюдения SLA;
- б) среднее время выполнения заявки;
- в) прогноз количества заявок на следующий день;
- г) процент успешных ремонтов.

Значения показателей формируются посредством вызовов методов `AnalyticsService`.

Ниже размещены два аналитических графика, реализованных через компонент `MatplotlibPanel`, который инкапсулирует `FigureCanvasQTAgg` и обеспечивает интеграцию библиотеки `Matplotlib` с `PyQt6`.

На странице показывается график количества заявок по дням за последние 21 день и горизонтальная диаграмма загрузки инженеров.

Использование горизонтальной диаграммы позволяет более наглядно отображать сравнительную загрузку сотрудников при большом количестве инженеров. (см. рисунок 4)

В нижней части страницы выводится текстовая сводка, которая содержит общее количество обработанных заявок, процент просроченных заявок, среднюю стоимость ремонта и среднюю продолжительность ремонта.

Все данные загружаются в рамках транзакции через `session_scope()`. При возникновении ошибок отображается диалоговое сообщение `QMessageBox.critical`, что обеспечивает базовый механизм информирования пользователя о сбоях.

2.5.2 Страница «Заявки»

Страница управления заявками (ui/pages/orders_page.py) предназначена для работы диспетчера и содержит таблицу сервисных заказов (QTableWidget).

Таблица включает следующие столбцы: идентификатор заявки, клиент, инженер, заголовок заявки, статус, приоритет, дата создания, дата завершения. Для повышения удобства работы реализованы фильтрация по статусу заявки, и полнотекстовый поиск по заголовку и описанию. Ключевыми операциями страницы являются назначение инженера и завершение заявки.

При назначении инженера пользователь выбирает заявку и сотрудника из выпадающего списка. Далее вызывается метод `OrderService.assign_engineer()`, после чего таблица автоматически обновляется.

Операция завершения заявки переводит её в статус `completed` и фиксирует время закрытия (`completed_at`). Все изменения выполняются внутри транзакции. При возникновении исключений выполняется автоматический откат изменений, а пользователь получает уведомление об ошибке. (см. рисунок 5)

	ID	Клиент	Инженер	Заголовок	Статус	Приоритет	Создана	Завершена
1	73	Иван Петров	Алексей ...	Не включается...	assigned	high	2026-05-12 ...	
2	74	ООО ТехноЛайн	Антон Громов	Замена матриц...	in_progress	medium	2026-05-12 ...	
3	75	Светлана ...	Виктор Хан	Диагностика ...	new	low	2026-05-12 ...	
4	70	Иван Петров	Алексей ...	Не включается...	assigned	high	2026-05-12 ...	
5	71	ООО ТехноЛайн	Антон Громов	Замена матриц...	in_progress	medium	2026-05-12 ...	
6	72	Светлана ...	Виктор Хан	Диагностика ...	new	low	2026-05-12 ...	
7	67	Иван Петров	Алексей ...	Не включается...	assigned	high	2026-05-12 ...	
8	68	ООО ТехноЛайн	Антон Громов	Замена матриц...	in_progress	medium	2026-05-12 ...	
9	69	Светлана ...	Виктор Хан	Диагностика ...	new	low	2026-05-12 ...	
10	64	Иван Петров	Алексей ...	Не включается...	assigned	high	2026-05-12 ...	
11	65	ООО ТехноЛайн	Антон Громов	Замена матриц...	in_progress	medium	2026-05-12 ...	
12	66	Светлана ...	Виктор Хан	Диагностика ...	completed	low	2026-05-12 ...	2026-05-12 ...
13	62	Лаборатория ...	Алексей ...	Импорт: ...	assigned	high	2026-05-12 ...	
14	63	ООО Документ	Антон Громов	Импорт: ...	assigned	medium	2026-05-12 ...	
15	59	Иван Петров	Алексей ...	Не включается...	assigned	high	2026-05-12 ...	
16	60	ООО ТехноЛайн	Антон Громов	Замена матриц...	completed	medium	2026-05-12 ...	2026-05-12 ...
17	61	Светлана ...	Виктор Хан	Диагностика ...	new	low	2026-05-12 ...	
18	45	ООО РемСити	Алексей ...	Заявка #45: ...	in_progress	high	2026-05-12 ...	
19	31	МУП ГорСвет	Павел Никитин	Заявка #31: ...	completed	critical	2026-05-12 ...	2026-05-14 ...
20	22	ООО Спектр	Екатерина ...	Заявка #22: ...	completed	low	2026-05-11 ...	2026-05-13 ...
21	48	Магазин Цифра	Денис Орлов	Заявка #48: ...	completed	medium	2026-05-02 ...	2026-05-04 ...
22	33	ТСЖ Речной	Алексей ...	Заявка #33: ...	completed	medium	2026-04-29 ...	2026-05-03 ...
23	29	Банк Партнёр	Мария Ким	Заявка #29: ...	completed	medium	2026-04-29 ...	2026-04-30 ...

Рисунок 5 – Страница «Заявки»

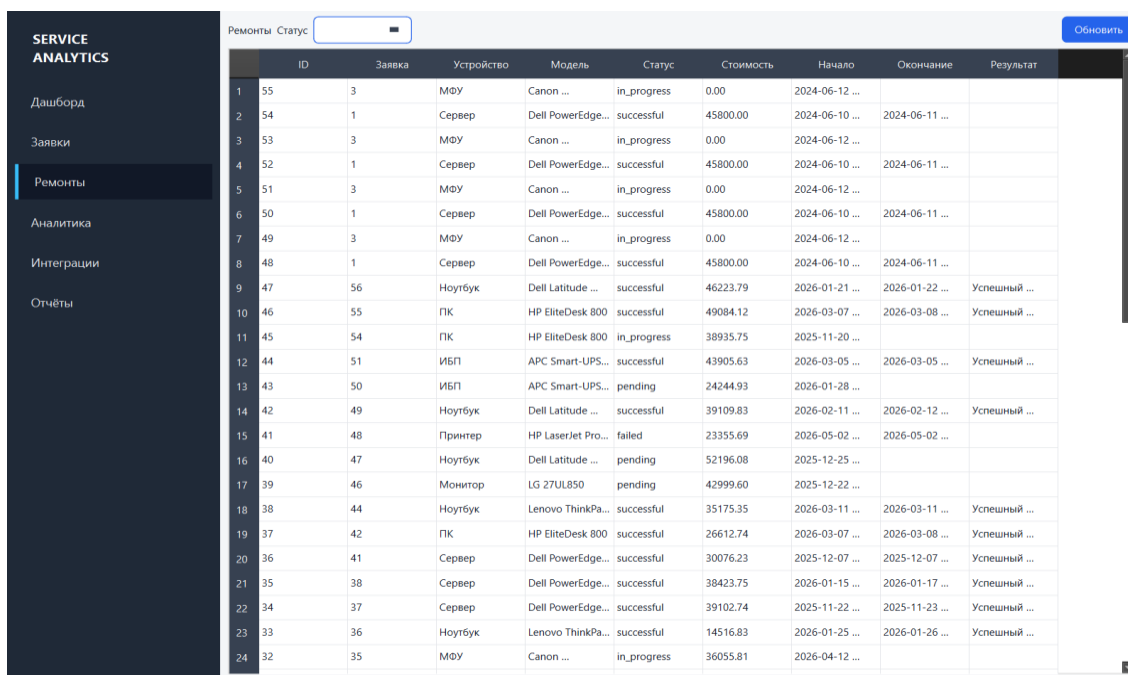
2.5.3 Страница «Ремонты»

Страница ремонтов (ui/pages/repairs_page.py) предназначена для просмотра информации о выполненных сервисных работах.

Таблица содержит следующие данные: идентификатор ремонта, номер сервисной заявки, тип устройства, модель оборудования, статус ремонта, стоимость работ, дата начала, дата завершения, результат выполнения.

Дополнительно реализована фильтрация по статусу ремонта.

В данном проекте интерфейс ориентирован преимущественно на аналитический просмотр данных. Функции создания и редактирования ремонтов отсутствуют, поскольку основной целью дипломного проекта является демонстрация аналитических возможностей системы. (см. рисунок 6)



	ID	Заявка	Устройство	Модель	Статус	Стоимость	Начало	Окончание	Результат
1	55	3	МФУ	Canon ...	in_progress	0.00	2024-06-12 ...		
2	54	1	Сервер	Dell PowerEdge...	successful	45800.00	2024-06-10 ...	2024-06-11 ...	
3	53	3	МФУ	Canon ...	in_progress	0.00	2024-06-12 ...		
4	52	1	Сервер	Dell PowerEdge...	successful	45800.00	2024-06-10 ...	2024-06-11 ...	
5	51	3	МФУ	Canon ...	in_progress	0.00	2024-06-12 ...		
6	50	1	Сервер	Dell PowerEdge...	successful	45800.00	2024-06-10 ...	2024-06-11 ...	
7	49	3	МФУ	Canon ...	in_progress	0.00	2024-06-12 ...		
8	48	1	Сервер	Dell PowerEdge...	successful	45800.00	2024-06-10 ...	2024-06-11 ...	
9	47	56	Ноутбук	Dell Latitude ...	successful	46223.79	2026-01-21 ...	2026-01-22 ...	Успешный ...
10	46	55	ПК	HP EliteDesk 800	successful	49084.12	2026-03-07 ...	2026-03-08 ...	Успешный ...
11	45	54	ПК	HP EliteDesk 800	in_progress	38935.75	2025-11-20 ...		
12	44	51	ИБП	APC Smart-UPS...	successful	43905.63	2026-03-05 ...	2026-03-05 ...	Успешный ...
13	43	50	ИБП	APC Smart-UPS...	pending	24244.93	2026-01-28 ...		
14	42	49	Ноутбук	Dell Latitude ...	successful	39109.83	2026-02-11 ...	2026-02-12 ...	Успешный ...
15	41	48	Принтер	HP LaserJet Pro...	failed	23355.69	2026-05-02 ...	2026-05-02 ...	
16	40	47	Ноутбук	Dell Latitude ...	pending	52196.08	2025-12-25 ...		
17	39	46	Монитор	LG 27UL850	pending	42999.60	2025-12-22 ...		
18	38	44	Ноутбук	Lenovo ThinkPa...	successful	35175.35	2026-03-11 ...	2026-03-11 ...	Успешный ...
19	37	42	ПК	HP EliteDesk 800	successful	26612.74	2026-03-07 ...	2026-03-08 ...	Успешный ...
20	36	41	Сервер	Dell PowerEdge...	successful	30076.23	2025-12-07 ...	2025-12-07 ...	Успешный ...
21	35	38	Сервер	Dell PowerEdge...	successful	38423.75	2026-01-15 ...	2026-01-17 ...	Успешный ...
22	34	37	Сервер	Dell PowerEdge...	successful	39102.74	2025-11-22 ...	2025-11-23 ...	Успешный ...
23	33	36	Ноутбук	Lenovo ThinkPa...	successful	14516.83	2026-01-25 ...	2026-01-26 ...	Успешный ...
24	32	35	МФУ	Canon ...	in_progress	36055.81	2026-04-12 ...		

Рисунок 6 – Страница «Ремонты»

2.5.4 Страница «Аналитика»

Страница аналитики (ui/pages/analytics_page.py) предоставляет расширенные средства анализа сервисной деятельности.

Верхняя часть страницы содержит текстовую сводку по соблюдению SLA, доле просроченных заявок, успешности ремонтов, средней стоимости и длительности работ.

Ниже отображаются графические элементы: круговая диаграмма распределения статусов ремонтов, гистограмма распределения длительности ремонтов, диаграмма нарушений SLA по приоритетам заявок.

Все графики формируются средствами Matplotlib и отображаются через компонент MatplotlibPanel.

Для актуализации информации предусмотрена кнопка «Обновить данные», инициирующая повторную загрузку аналитических показателей из базы данных.

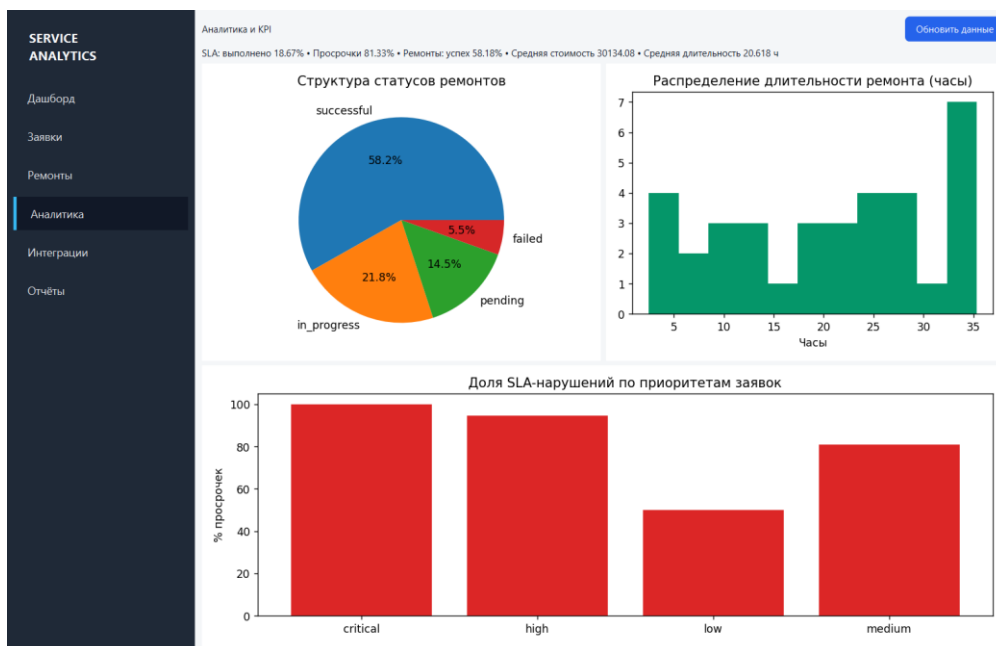


Рисунок 7 – Страница «Аналитика»

2.5.5 Страница «Интеграции»

Страница интеграций (`ui/pages/integrations_page.py`) предназначена для управления ETL-процессами и контроля операций синхронизации данных.

На панели инструментов расположены три основные кнопки: «Синхронизация Bitrix24», «Выгрузка из 1С», «Импорт Excel».

Кнопка синхронизации Bitrix24 запускает `BitrixSyncService.run()`, который инициирует ETL-пайплайн загрузки сервисных заявок. Загрузка данных из 1С выполняется через `OneCSyncService.run()` и предназначена для импорта информации о ремонтах и финансовых данных. Импорт Excel реализован через диалог выбора файла и вызов сервиса `ExcelImportService.import_orders_workbook()`. Нижняя часть страницы содержит две таблицы: журнал интеграций (`integration_logs`) и история синхронизаций (`sync_history`).

Журнал интеграций отображает источник данных, тип операции, статус выполнения, текст сообщения, время выполнения. Таблица истории синхронизаций содержит время начала и окончания операции, количество обработанных записей и итоговый статус синхронизации.

Наличие отдельного интерфейса управления интеграциями упрощает контроль ETL-процессов и позволяет оперативно выявлять ошибки загрузки данных. (рисунок 8)

SERVICE ANALYTICS

Дашборд

Заявки

Ремонты

Аналитика

Интеграции

Отчёты

Интеграции и ETL

Синхронизация Bitrix24

Выгрузка из 1С

Импорт Excel

Обновить журналы

Журнал интеграций

	ID	Система	Операция	Статус	Сообщение
1	15	onec	sync_repairs	success	processed_in=2...
2	13	bitrix24	orders_etl	success	loaded=3
3	14	bitrix24	sync_orders	success	processed_in=3...
4	12	onec	sync_repairs	success	processed_in=2...
5	10	bitrix24	orders_etl	success	loaded=3
6	11	bitrix24	sync_orders	success	processed_in=3...
7	9	onec	sync_repairs	success	processed_in=2...
8	7	bitrix24	orders_etl	success	loaded=3
9	8	bitrix24	sync_orders	success	processed_in=3...
10	5	bitrix24	orders_etl	success	loaded=3
11	6	bitrix24	sync_orders	success	processed_in=3...

История синхронизаций

	ID	Источник	Обработано	Начало	Окончание	Статус
1	9	onec	2	2026-05-16 ...	2026-05-16 ...	success
2	8	bitrix24	3	2026-05-12 ...	2026-05-12 ...	success
3	7	onec	2	2026-05-12 ...	2026-05-12 ...	success
4	6	bitrix24	3	2026-05-12 ...	2026-05-12 ...	success
5	5	onec	2	2026-05-12 ...	2026-05-12 ...	success
6	4	bitrix24	3	2026-05-12 ...	2026-05-12 ...	success
7	3	bitrix24	3	2026-05-12 ...	2026-05-12 ...	success
8	2	onec	2	2026-05-12 ...	2026-05-12 ...	success
9	1	bitrix24	3	2026-05-12 ...	2026-05-12 ...	success

Рисунок 8 – Страница «Интеграции»

2.5.6 Страница «Отчёты»

Страница отчётов (ui/pages/reports_page.py) предназначена для экспорта аналитической информации во внешние форматы.

Интерфейс содержит две основные операции, это экспорт в Excel и экспорт в PDF.

При экспорте Excel формируется многостраничный файл, включающий в себя управленческую сводку, показатели SLA, статистику ремонтов, распределение заявок по дням, загрузку инженеров, результаты KPI-анализа.

Экспорт PDF формирует документ с титульной страницей и аналитическими графиками. Для генерации отчётов используются сервисы ReportingService, ExcelAnalyticsReport и возможности библиотеки Matplotlib.

После успешного завершения экспорта пользователю отображается информационное сообщение с указанием пути сохранённого файла. (рисунок 9)

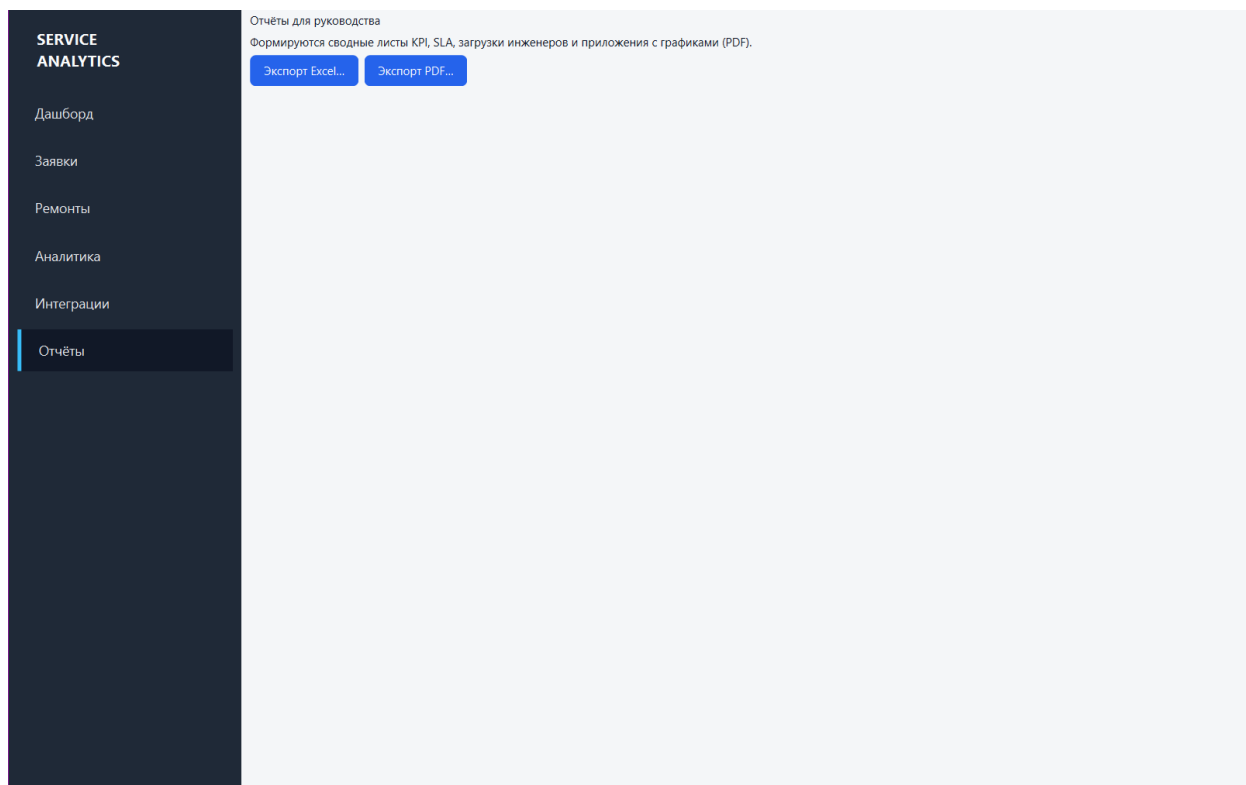


Рисунок 9 – Страница «Отчёты»

2.5.7 Стилизация интерфейса

Визуальное оформление системы реализовано посредством глобальной таблицы стилей QSS, определённой в файле `ui/styles.py`.

Основные элементы оформления включают в себя тёмную боковую панель (`#1f2937`), навигационные кнопки с эффектами наведения и выделения, стилизованные таблицы с чередованием строк, кнопки действий синего цвета, поля ввода со скруглёнными углами и визуальным выделением при фокусе.

Для обеспечения единообразного отображения интерфейса на различных операционных системах используется стиль `Fusion`, подключаемый через `QStyleFactory.create("Fusion")`.

Применение единой системы стилей позволило сформировать целостный корпоративный интерфейс, обеспечивающий удобство работы и визуальную согласованность всех компонентов приложения.

Таким образом, пользовательский интерфейс системы обеспечивает доступ ко всем ключевым функциям программного комплекса: управлению заявками, анализу сервисных показателей, контролю интеграций и формированию отчётности. Архитектурное разделение интерфейса на независимые страницы упрощает сопровождение системы и создаёт основу для дальнейшего расширения функциональности.

2.6 Интеграция с внешними сервисами

Одной из ключевых задач разрабатываемой системы является консолидация данных из разнородных источников. В большинстве сервисных организаций информация о клиентах, ремонтах, финансовых операциях и заявках распределена между несколькими системами, что затрудняет получение целостной аналитической картины. Для решения данной проблемы в программном комплексе реализован механизм интеграции с внешними источниками данных.

Система поддерживает работу с тремя типами источников: CRM-системой Bitrix24, системой учёта 1С, Excel-файлами.

Все интеграции реализованы через единый ETL-пайплайн (Extract → Transform → Load), что обеспечивает унификацию процессов загрузки, преобразования и сохранения данных.

2.6.1 Интеграция с Bitrix24

Модуль интеграции с Bitrix24 расположен в пакете `integrations/bitrix24` и предназначен для загрузки сервисных заявок из CRM-системы.

Взаимодействие с внешним API реализовано в классе `BitrixMockClient` (`bitrix_client.py`). В демонстрационной версии проекта используется механизм имитации REST API, позволяющий тестировать работу системы без подключения к реальному серверу Bitrix24.

Клиент имитирует вызов метода `crm.deal.list` и возвращает заранее подготовленный JSON-массив заявок, сохранённый в переменной `MOCK_DEALS_JSON`.

Если в системе определена переменная окружения `BITRIX_WEBHOOK_URL`, выполняется реальный HTTP-запрос к REST API Bitrix24 через вебхук. При отсутствии вебхука или возникновении ошибки система автоматически переключается на использование mock-данных. Такой подход позволяет обеспечить работоспособность приложения независимо от наличия внешнего подключения.

Структура импортируемых данных включает в себя идентификатор заявки, заголовок, описание проблемы, приоритет, статус и данные клиента (имя, телефон, электронная почта). Извлечение данных выполняет компонент `BitrixExtractor` (`etl/extractor.py`), который получает данные от клиента и преобразует их в объект `RawBatch`. Оркестрация процесса синхронизации реализована в сервисе `BitrixSyncService` (`bitrix_sync.py`). Последовательность работы включает в себя создание записи в таблице `sync_history`, запуск ETL-пайплайна, обновление статуса синхронизации, а также запись результата в журнал интеграций.

В случае возникновения ошибки статус синхронизации изменяется на `failed`, а информация о сбое сохраняется в журнале.

2.6.2 Интеграция с 1С

Интеграция с системой 1С реализована через модуль `integrations/ones` и предназначена для загрузки данных о ремонтах, стоимости работ и оборудовании.

В демонстрационной версии используется класс `OneCMockClient` (`ones_client.py`), который выполняет чтение данных из файлов `mock_repairs.csv` и `mock_repairs.json`. При отсутствии файлов система использует встроенный JSON-набор данных.

Структура записей включает в себя ссылку на заявку, тип устройства, модель оборудования, описание неисправности, статус ремонта, дату начала и завершения, а также стоимость работ.

Для приведения структуры данных к внутреннему формату используется `mapper bundle_to_repairs` (`ones_mapper.py`). На этапе преобразования выполняется унификация названий полей, например таких, как `DEVICE` → `device_type`, `MODEL` → `device_model`, `STATUS` → `repair_status`. Извлечение данных осуществляется компонентом `OneCExtractor`, возвращающим объект `RawBatch`.

Процесс синхронизации реализован сервисом `OneCSyncService`, который сначала выполняет запуск ETL-пайплайна, потом сохранение данных в БД, затем обновление `sync_history` и логирование результатов операции. Архитектурно интеграция с 1С построена аналогично интеграции с Bitrix24, что обеспечивает единообразие обработки внешних источников данных.

2.6.3 Импорт данных в Excel

Дополнительным механизмом загрузки данных является импорт Excel-файлов. Такой способ интеграции особенно актуален для небольших организаций, в которых часть информации хранится в табличных документах. Импорт реализован через компонент ExcelExtractor (etl/extractor.py), использующий функцию `pandas.read_excel()`.

Для корректной обработки файл должен содержать следующие столбцы:

- а) `external_id`;
- б) `title`;
- в) `description`;
- г) `priority`;
- д) `status`;
- е) `client_name`;
- ж) `client_phone`;
- з) `client_email`.

После чтения данные преобразуются в единый внутренний формат ETL-пайплайна.

Для упрощения демонстрации работы системы предусмотрен скрипт `create_sample_orders_workbook.py`, автоматически формирующий пример Excel-файла с тестовыми заявками.

Импорт данных инициируется сервисом `ExcelImportService` (`excel_importer.py`), который представляет собой обёртку над методом `ETLPipeline.run_excel_orders()`. После завершения импорта информация об операции автоматически фиксируется в журнале интеграций.

2.6.4 Логирование и аудит интеграций

Для обеспечения контроля ETL-процессов и возможности последующего аудита в системе реализован механизм журналирования операций интеграции. Информация о загрузках сохраняется в двух таблицах: `integration_logs` и `sync_history`. Таблица `integration_logs` содержит сведения о каждой операции, такой как источник данных, тип операции, статус выполнения, текст сообщения, время выполнения. Таблица `sync_history` хранит агрегированную информацию о сеансах синхронизации: время начала, время завершения, количество обработанных записей, итоговый статус.

Разделение журналирования на уровень операций и уровень сеансов синхронизации позволяет отслеживать ошибки ETL-процессов, проанализировать производительность загрузки, контролировать стабильность интеграций и проводить аудит обмена данными.

Работа с журналами реализована через репозитории `IntegrationLogRepository` и `SyncHistoryRepository`, предоставляющие методы добавления и получения последних записей.

В пользовательском интерфейсе журналы отображаются на странице «Интеграции», что позволяет оператору контролировать состояние загрузок без обращения к базе данных или серверным журналам.

Таким образом, реализованный механизм интеграции обеспечивает централизованную загрузку данных из внешних систем, унификацию форматов хранения и возможность последующего аналитического анализа информации в рамках единой платформы.

2.7 Обеспечение информационной безопасности

Разрабатываемая система предназначена для эксплуатации внутри сервисной организации и функционирует в изолированном контуре без прямого доступа из сети Интернет. Несмотря на ограниченный характер использования, при проектировании были рассмотрены базовые меры обеспечения информационной безопасности, необходимые для защиты данных и снижения эксплуатационных рисков.

Одним из ключевых аспектов является разграничение доступа пользователей к функциональным возможностям системы. В текущей реализации механизм ролевой модели отсутствует, поскольку предполагается, что приложение используется на защищённой рабочей станции администратора или диспетчера, доступ к которой контролируется средствами операционной системы. Вместе с тем архитектура программного решения допускает последующее внедрение аутентификации и разграничения прав доступа. В частности, административные разделы, связанные с настройкой интеграций и формированием отчётности, могут быть доступны только пользователям с повышенными привилегиями, тогда как работа с заявками — диспетчерам и операторам. Реализация такого подхода возможна на уровне сервисного слоя посредством проверки полномочий через специализированные декораторы или middleware-компоненты.

Отдельное внимание должно быть уделено защите персональных данных. В базе данных системы содержатся сведения о клиентах, включая их имена, номера телефонов и адреса электронной почты. В демонстрационной версии проекта эти данные сохраняются в открытом виде, что допустимо в рамках учебного прототипа, однако не соответствует требованиям промышленной эксплуатации. При внедрении системы в реальную среду необходимо обеспечить криптографическую защиту конфиденциальной информации. Наиболее целесообразным решением является применение симметричного шифрования, например алгоритма AES-256, на уровне прикладной логики. При этом ключи шифрования должны храниться

отдельно от основной базы данных, что позволяет снизить риск компрометации информации при несанкционированном доступе к СУБД.

Для обеспечения корректной работы интеграционных механизмов в системе реализовано ведение журналов событий. В них фиксируются операции, связанные с импортом и обработкой данных, поступающих из внешних источников. Такой подход обеспечивает возможность последующего анализа и аудита выполняемых действий, а также соответствует базовым требованиям по регистрации событий безопасности в информационных системах, обрабатывающих персональные данные.

Отдельное внимание следует уделить поддержанию актуальности программной среды. Рабочие станции, на которых эксплуатируется система, должны быть обеспечены лицензионным антивирусным программным обеспечением. Кроме того, необходимо регулярное обновление операционной системы и используемых библиотек Python. Своевременная установка обновлений снижает риск эксплуатации известных уязвимостей и уменьшает вероятность компрометации или нарушения работы системы вследствие использования устаревших программных компонентов.

2.8 Тестирование

Для обеспечения корректной работы системы в рамках проекта реализован набор модульных тестов, размещённых в директории tests. В качестве примера используется файл test_sla_calculator.py, в котором сосредоточены проверки логики расчёта SLA-показателей.

При разработке тестов основное внимание уделялось корректности выполнения вычислений и обработке временных параметров. В частности, предусмотрен сценарий, проверяющий фиксацию нарушения SLA для просроченной, но всё ещё открытой заявки; данный случай был рассмотрен в разделе 2.4.1.

Дополнительно реализован тест, подтверждающий корректное поведение системы при обработке заявки, завершённой в пределах установленного срока:

```
def test_sla_met_when_completed_within_target():
    calc = SLACalculator()
    created = datetime(2025, 1, 1, 8, 0,
tzinfo=timezone.utc)
    completed = created + timedelta(hours=10)
    row = OrderSLAInput(
        order_id=1,
        priority="high",
        status="completed",
        created_at=created,
        completed_at=completed
    )
    res = calc.evaluate_order(row, now=completed)
    assert res.breached is False
```

Данный тест моделирует ситуацию, при которой заявка с высоким приоритетом была закрыта до истечения допустимого времени обработки. Результаты выполнения подтверждают, что алгоритм корректно интерпретирует подобные случаи и не фиксирует нарушение SLA при соблюдении установленных ограничений.

Следует отметить, что используемые тесты не зависят от внешних компонентов инфраструктуры. Для формирования тестовых сценариев применяются фиктивные временные метки и искусственно создаваемые

входные данные, что позволяет выполнять проверки без подключения к базе данных. Запуск тестов осуществляется с использованием фреймворка `pytest`.

В текущей версии проекта покрытие тестами ограничено ключевыми вычислительными модулями. Проверка репозитория, сервисного слоя и интеграционных сценариев в полном объёме не реализована, что рассматривается как направление дальнейшего развития системы. В практической эксплуатации рекомендуется дополнительно протестировать следующие компоненты:

- а) метод `OrderService.assign_engineer` – корректность изменения статуса заявки и обновления временных меток;
- б) метод `ETLTransformer.transform_orders` – правильность нормализации статусов, поступающих из различных внешних источников;
- в) метод `EngineerKPIAggregator.success_rates` – точность вычисления процентных показателей эффективности инженеров.

Для реализации интеграционного тестирования целесообразно использовать временную тестовую базу данных, создаваемую автоматически перед запуском тестов. В качестве такого решения может применяться `SQLite` в режиме `in-memory`, что позволяет выполнять проверки быстро и без влияния на основную рабочую базу данных.

3 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

Внедрение корпоративной системы аналитики выездного сервиса и ремонта связано с необходимостью первоначальных инвестиций в разработку, настройку и сопровождение программного обеспечения. В связи с этим требуется оценить экономическую целесообразность реализации проекта, определить предполагаемый эффект от автоматизации бизнес-процессов, а также сравнить собственную разработку с альтернативным вариантом приобретения готового программного решения.

В данной главе выполнен расчёт затрат на создание системы, проведена оценка ожидаемого экономического эффекта, рассчитаны ключевые показатели инвестиционной эффективности и рассмотрены основные риски внедрения.

3.1 Расчёт затрат на разработку

Для оценки затрат на разработку программного обеспечения рассматриваются следующие параметры: команда из четырех разработчиков со средней зарплатой 70 000 рублей в месяц, период разработки в 3,5 месяца и аренда сервера за 2 000 рублей в месяц. Также учитываются страховые взносы и накладные расходы(см. таблицу 3).

1) Фонд оплаты труда (ФОТ) рассчитывается так:

$$\text{ФОТ} = \text{ЗП} \times \text{Ч} \times \text{Т}$$
 ((ЗП – ежемесячная заработная плата одного сотрудника, руб.; Ч – численность сотрудников, чел.; Т – продолжительность разработки, мес.)
$$70\,000 \text{ рублей (зарплата)} \times 4 \text{ (разработчика)} \times 3,5 \text{ (месяца)} = 980\,000 \text{ рублей.}$$

2) Страховые взносы составят 30% от ФОТ:

$$\text{СВ} = \text{ФОТ} \times 30\% \text{ (СВ – страховой взнос)}$$
$$980\,000 \text{ рублей} \times 0,3 = 294\,000 \text{ рублей.}$$

3) Накладные расходы, включая электроэнергию и амортизацию, примем в размере 25% от ФОТ:

$$\text{НР} = \text{ФОТ} \times 0,25 \text{ (НР – накладные расходы)}$$
$$980\,000 \text{ рублей} \times 0,25 = 245\,000 \text{ рублей.}$$

4) Аренда сервера за 3,5 месяца:

$$2\,000 \text{ рублей} \times 3,5 = 7\,000 \text{ рублей.}$$

5) Общая себестоимость равна сумме всех расходов:

$$\text{С} = \text{ФОТ} + \text{СВ} + \text{НР} + \text{Сервер} \text{ (С – себестоимость)}$$
$$980\,000 \text{ (ФОТ)} + 294\,000 \text{ (страховые взносы)} + 245\,000 \text{ (накладные расходы)} \\ + 7\,000 \text{ (сервер)} = 1\,526\,000 \text{ рублей.}$$

Итоговая себестоимость разработки программного обеспечения составляет 1 526 000 рублей.

Таблица 3 – расчет затрат на разработку

№ п/п	Статья затрат	Сумма, руб.
	Фонд оплаты труда	
	Страховые взносы	
	Накладные расходы	
	Аренда сервера	
	Итого	

Полученные результаты показывают, что основную часть себестоимости составляют затраты на оплату труда разработчиков, что характерно для программных продуктов, основанных преимущественно на интеллектуальном труде. Затраты на инфраструктуру при этом остаются относительно невысокими, однако при реальном внедрении могут возрасти при использовании более мощного оборудования или лицензионного ПО.

3.2 Оценка экономического эффекта от внедрения

Для оценки экономической целесообразности внедрения разработанной системы необходимо определить ожидаемый экономический эффект.

Исходные данные(см. таблицу 4):

Таблица 4 – исходные данные для расчета экономической эффективности

№	Показатель	Значение
	Ежемесячный объём заявок	
	Численность технических специалистов	8 чел.
	Средняя прибыль с одной заявки	2 500 руб.
	Годовой объём заявок	
	Себестоимость разработки	1 526 000 руб.

Расчёт экономического эффекта

1) Прирост прибыли за счёт увеличения количества заявок

Предполагается, что внедрение системы позволит увеличить объём выполненных заявок на 15 % за счёт сокращения потерь и оптимизации процессов (см. таблицу 5).

Новый объём:

1380 заявок в месяц ($1200 * 1,15$).

Дополнительные заявки:

180 в месяц ($1380 - 1200$).

Дополнительная месячная прибыль:

450 000 руб. ($180 * 2500$).

Дополнительная годовая прибыль:

5 400 000 руб. ($450\,000 * 12$).

2) Экономия от оптимизации работы

Допустим, автоматизация сократит административные расходы на зарплату одного сотрудника (например, диспетчера) в 50 000 руб./месяц.

Годовая экономия:

600 000 руб. ($50\,000 * 12$).

3) Общий годовой экономический эффект:

6 000 000 руб. ($5\,400\,000 + 600\,000$).

4) Чистый экономический эффект:

4 474 000 руб. ($6\,000\,000 - 1\,526\,000$).

5) Срок окупаемости

Приблизительно 0,25 года, или 3 месяца ($1\,526\,000 / 6\,000\,000$).

6. Коэффициент экономической эффективности

Около 3,93 ($6\,000\,000 / 1\,526\,000$). Это значит, что каждый вложенный рубль приносит около 4 рублей прибыли.

Таблица 5 – расчет годового экономического эффекта

№ п/п	Показатель	Значение
	Дополнительные заявки в месяц	
	Дополнительная прибыль в месяц	450 000 руб.
	Дополнительная прибыль в год	5 400 000 руб.

Продолжение таблицы 5

	Экономия на зарплате	600 000 руб.
	Общий годовой эффект	6 000 000 руб.

3.3 Оценка рисков проекта

Разработка и внедрение программного продукта представляет собой инвестиционный проект, связанный с определёнными рисками. Наличие рисков может повлиять на сроки реализации, бюджет проекта и величину ожидаемого экономического эффекта.

Оценка рисков позволяет заранее определить возможные проблемы и разработать меры по их минимизации.

Основные виды рисков проекта(см таблицу 6)

1) Технические риски: ошибки в программном коде; несовместимость программных компонентов; нестабильная работа серверного оборудования; недостаточная производительность системы.

Экономические последствия: увеличение сроков разработки; рост фонда оплаты труда; дополнительные расходы на доработку.

2) Организационные риски: несогласованность действий команды; изменение требований заказчика; недостаточная квалификация сотрудников; нарушение сроков выполнения задач.

Экономические последствия: перерасход бюджета; увеличение накладных расходов; снижение ожидаемой прибыли.

3) Финансовые риски: увеличение стоимости разработки; недостижение планируемого роста прибыли; превышение срока окупаемости; необходимость дополнительных инвестиций.

4) Юридические риски: нарушением законодательства о персональных данных; неправомерным использованием программного обеспечения; несоблюдением требований информационной безопасности.

Экономические последствия: штрафные санкции; судебные издержки; репутационные потери.

Таблица 6 – Анализ рисков разработки и внедрения

№ п/п	Вид риска	Вероятность	Возможные последствия	Меры минимизации
-------	-----------	-------------	-----------------------	------------------

Продолжение таблицы 6

1	Технический	Средняя	Увеличение затрат и сроков	Тестирование, резервное копирование
2	Организационный	Средняя	Перерасход бюджета	Чёткое планирование, контроль сроков
3	Финансовый	Низкая	Снижение прибыли	Детальный расчет эффективности
4	Юридический	Низкая	Штрафы	Соблюдение законодательства

Даже при возможном снижении ожидаемого экономического эффекта на 20%, проект остаётся прибыльным.

Если годовой эффект уменьшится с 6 000 000 руб. до:

$$6000000 \times 0,8 = 4800000 \text{руб.}$$

Срок окупаемости составит:

$$1526000 / 4800000 \approx 0,32 \text{года}$$

Это примерно 4 месяца.

Таким образом, даже при неблагоприятном сценарии проект остаётся экономически целесообразным.

3.4 Сравнение разработки собственного продукта с покупкой готового решения

При принятии решения о внедрении информационной системы предприятие может выбрать один из двух вариантов:

- а) Разработка собственного программного продукта
- б) Приобретение готового коммерческого решения

Для обоснования целесообразности выбранного варианта необходимо провести сравнительный экономический анализ(см. таблицу 7).

1) Вариант приобретения готового решения

Рассмотрим условный вариант приобретения корпоративной системы уровня SAP или Microsoft Dynamics 365 Field Service.

Как правило, подобные решения предполагают покупку лицензий, оплату внедрения и ежегодную техническую поддержку, а также, дополнительную настройку под специфику предприятия.

Предположительные затраты:

Лицензия на 10 пользователей — 1 200 000 руб.

Внедрение и настройка — 800 000 руб.

Обучение персонала — 200 000 руб.

Годовая техническая поддержка — 300 000 руб.

Итого первоначальные затраты:

$$1200000+800000+200000=2200000\text{руб.}$$

С учетом поддержки в первый год:

$$2200000+300000=2500000\text{руб.}$$

2) Вариант собственной разработки

Согласно произведенным расчетам себестоимость разработки составила:

$$1\,526\,000\text{руб.}$$

Дополнительных лицензионных платежей не требуется, так как система разрабатывается для внутреннего использования.

Таблица 7 – сравнение вариантов внедрения

№ п/п	Показатель	Готовое решение	Собственная разработка
1	Первоначальные затраты	2 500 000 руб.	1 526 000 руб.
2	Ежегодные лицензионные платежи	Да	Нет
3	Адаптация под специфику	Ограниченная	Полная
4	Зависимость от поставщика	Высокая	Отсутствует
5	Гибкость развития	Средняя	Высокая

3) Экономическая разница вариантов

Разница первоначальных затрат:

$$2500000 - 1526000 = 974000 \text{ руб.}$$

Таким образом, собственная разработка дешевле почти на 1 миллион рублей уже на этапе внедрения. Кроме того, при использовании готового решения компания ежегодно несёт расходы на поддержку и лицензии, тогда как собственная система требует лишь затрат на сопровождение, которые значительно ниже стоимости коммерческих лицензий.

- а) Стратегические преимущества собственной разработки с экономической точки зрения собственная разработка позволяет:
- б) полностью адаптировать систему под бизнес-процессы компании,
- в) избежать переплаты за избыточный функционал,
- г) снизить долгосрочные эксплуатационные расходы,
- д) обеспечить независимость от внешнего поставщика,
- е) гибко модернизировать систему без покупки дополнительных лицензий.

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломного проекта разработана корпоративная информационно-аналитическая система, предназначенная для сопровождения процессов выездного сервисного обслуживания и ремонта оборудования. Созданное решение обеспечивает централизованный сбор, обработку и визуализацию данных о заявках, ремонтах и показателях эффективности сервисного подразделения. Поставленные в рамках исследования задачи выполнены в полном объёме.

На первом этапе работы был проведён анализ предметной области. В ходе исследования определён типовой жизненный цикл сервисной заявки — от регистрации обращения до закрытия ремонта и формирования итоговой отчётности. Кроме того, выделены основные категории пользователей системы и сформулированы требования к расчёту показателей SLA и KPI, используемых для оценки качества сервисного обслуживания и загрузки инженерного персонала.

Для обоснования необходимости разработки собственной системы выполнен сравнительный анализ существующих решений, включая SAP Field Service Management, Microsoft Dynamics 365 Field Service и GLPI. Проведённая оценка показала, что коммерческие платформы обладают широким функционалом, однако их внедрение и сопровождение требуют значительных финансовых затрат, что делает такие решения избыточными для предприятий среднего масштаба. В свою очередь, open-source системы предполагают существенную адаптацию под специфику сервисной деятельности. С учётом выявленных ограничений была подтверждена целесообразность разработки специализированного программного продукта.

В качестве технологической основы системы выбран стек, включающий Python, MySQL, SQLAlchemy, PyQt6, pandas и matplotlib. Данный набор технологий обеспечивает кроссплатформенность, сравнительно высокую скорость разработки и достаточную производительность при обработке аналитических данных. Использование Python позволило реализовать как

серверную бизнес-логику, так и инструменты аналитики в рамках единой программной среды.

В процессе проектирования разработана реляционная модель базы данных, включающая восемь ключевых сущностей: клиенты, инженеры, сервисные заявки, ремонты, запчасти, KPI-срезы и журналы интеграций. Для взаимодействия с базой данных реализован ORM-слой на основе SQLAlchemy 2.x с применением декларативного подхода. Архитектура серверной части построена с использованием паттернов Repository и Service, благодаря чему бизнес-логика была отделена от механизмов доступа к данным. Подобное решение повысило сопровождаемость кода и упростило дальнейшее расширение функциональности системы. Транзакционная целостность операций обеспечена посредством контекстного менеджера `session_scope()`.

Отдельное внимание уделено интеграции внешних источников данных. Разработаны ETL-процессы для импорта информации из Bitrix24, 1С и Excel-файлов. Реализованы алгоритмы расчёта SLA с учётом приоритетов заявок, а также механизмы агрегации KPI инженерного персонала. В частности, для оценки совокупной загрузки инженеров применён интегральный показатель `workload_score`, учитывающий количество выполненных заявок и ремонтов. Дополнительно реализованы модули прогнозирования спроса на сервисные работы методом скользящего среднего и инструменты дескриптивного статистического анализа ремонтов.

Пользовательский интерфейс системы разработан на базе PyQt6 и включает шесть функциональных разделов: дашборд, управление заявками, учёт ремонтов, аналитический модуль, подсистему интеграций и отчётность. Реализованы средства экспорта результатов анализа в форматы Excel и PDF с автоматическим построением графиков и диаграмм. Для проверки корректности вычислений проведено unit-тестирование ключевых компонентов системы, в том числе модуля расчёта SLA.

Экономическая эффективность проекта также получила количественное подтверждение. Согласно выполненным расчётам, себестоимость разработки

составила 1,53 млн рублей, тогда как ожидаемый годовой экономический эффект оценивается в 6,0 млн рублей. Расчётный срок окупаемости проекта не превышает трёх месяцев. Даже при снижении прогнозируемого эффекта на 20% внедрение системы остаётся экономически целесообразным. Дополнительно установлено, что собственная разработка обходится на 0,97 млн рублей дешевле по сравнению с внедрением готового корпоративного решения и не требует регулярных лицензионных платежей.

Таким образом, в ходе выполнения дипломного проекта была создана полнофункциональная система аналитики сервисного обслуживания, ориентированная на практическое применение в организациях, осуществляющих выездной ремонт и техническую поддержку оборудования. Полученные результаты подтверждают возможность её внедрения на предприятиях различного профиля. В дальнейшем развитие системы может быть связано с расширением ролевой модели доступа, внедрением механизмов шифрования персональных данных и использованием более сложных методов прогнозной аналитики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Гражданский кодекс Российской Федерации (часть четвертая) : федеральный закон от 18.12.2006 № 230-ФЗ (ред. от 30.04.2021). – Москва : Юридическая литература, 2021. – 78 с.
2. Российская Федерация. Об информации, информационных технологиях и о защите информации : федеральный закон от 27.07.2006 № 149-ФЗ (ред. от 02.07.2021). – Москва : Собрание законодательства РФ, 2006. – Ст. 31.
3. Российская Федерация. О персональных данных : федеральный закон от 27.07.2006 № 152-ФЗ (ред. от 30.12.2020). – Москва : Собрание законодательства РФ, 2006. – Ст. 34.
4. Российская Федерация. О коммерческой тайне : федеральный закон от 29.07.2004 № 98-ФЗ (ред. от 18.04.2018). – Москва : Собрание законодательства РФ, 2004. – Ст. 31.
5. Российская Федерация. Постановление Правительства РФ от 16.11.2015 № 1236 «Об установлении запрета на допуск программного обеспечения, происходящего из иностранных государств, для целей осуществления закупок для обеспечения государственных и муниципальных нужд». – Москва : Собрание законодательства РФ, 2015. – № 47. – Ст. 6606.
6. Российская федерация. Постановление Правительства РФ от 1 ноября 2012 г. N 1119 "Об утверждении требований к защите персональных данных при их обработке в информационных системах персональных данных" – Москва: Собрание законодательства РФ, 2007 - №48. – Ст. 6001.
7. ISO/IEC 27001:2013 Information Security Management Systems.
8. Российская федерация. Методические документы ФСТЭК России по обеспечению безопасности персональных данных при их обработке в ИСПДн.

9. Российская федерация. ГОСТ 7.32-2017. Отчёт о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017. – 28 с.
10. Российская федерация. ГОСТ Р 56939-2016. Защита информации. Общие положения.
11. SAP SE. SAP Field Service Management [Электронный ресурс]. – Режим доступа: <https://www.sap.com/products/field-service-management.html>
12. Microsoft Corporation. Microsoft Dynamics 365 Field Service [Электронный ресурс]. – Режим доступа: <https://dynamics.microsoft.com/ru-ru/field-service/>
13. GLPI Project. GLPI – Open Source IT Asset Management [Электронный ресурс]. – Режим доступа: <https://glpi-project.org>
14. Python Software Foundation. Python Documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/>
15. SQLAlchemy Authors. SQLAlchemy 2.0 Documentation [Электронный ресурс]. – Режим доступа: <https://docs.sqlalchemy.org/en/20/>
16. The Qt Company. Qt for Python (PyQt6) Documentation [Электронный ресурс]. – Режим доступа: <https://www.riverbankcomputing.com/static/Docs/PyQt6/>
17. pandas development team. pandas documentation [Электронный ресурс]. – Режим доступа: <https://pandas.pydata.org/docs/>
18. Matplotlib development team. Matplotlib documentation [Электронный ресурс]. – Режим доступа: <https://matplotlib.org/stable/contents.html>
19. Bitrix Inc. Bitrix24 REST API documentation [Электронный ресурс]. – Режим доступа: <https://apidocs.bitrix24.com>
20. Министерство цифрового развития, связи и массовых коммуникаций РФ. Реестр отечественного программного обеспечения [Электронный ресурс]. – Режим доступа: <https://reestr.digital.gov.ru/>
21. Федеральная служба государственной статистики. Цифровая экономика [Электронный ресурс]. – Режим доступа: <https://rosstat.gov.ru/statistics>

22. Методические рекомендации по организации выполнения и защиты выпускной квалификационной работы (дипломный проект) в КМПО РАНХиГС. – Москва, 2024.

ПРИЛОЖЕНИЕ А

Диаграмма развертывания

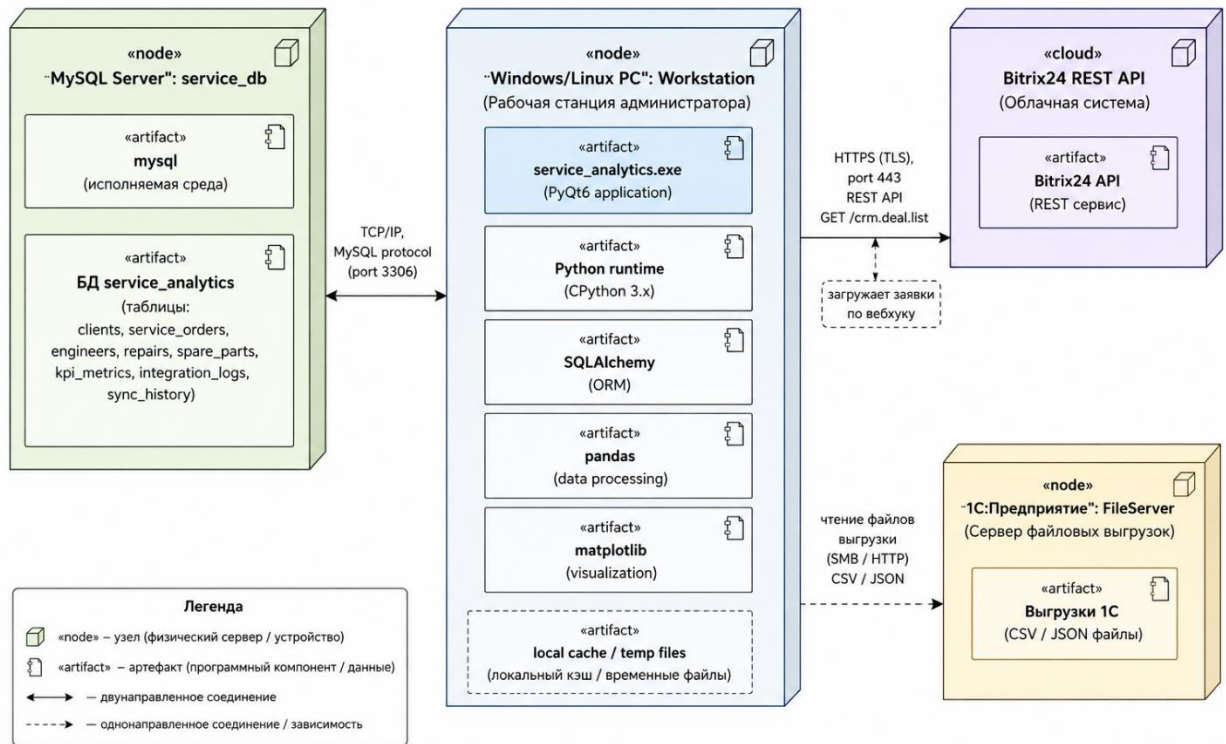
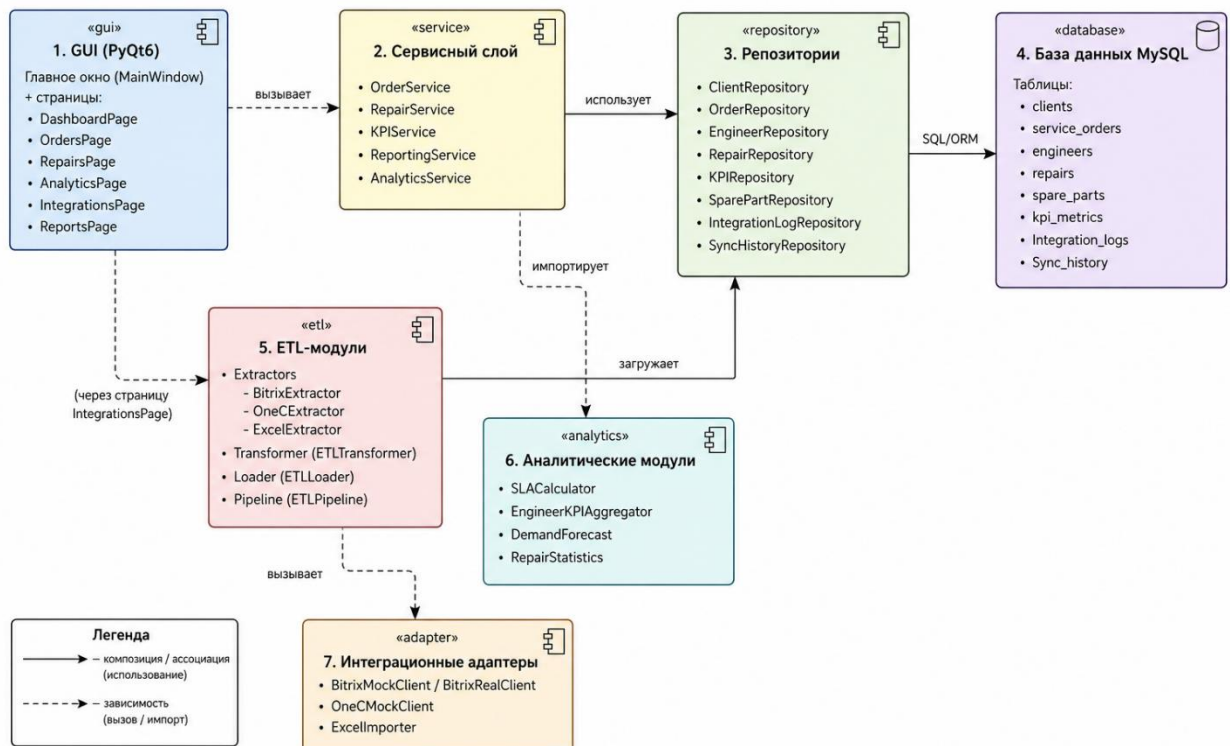
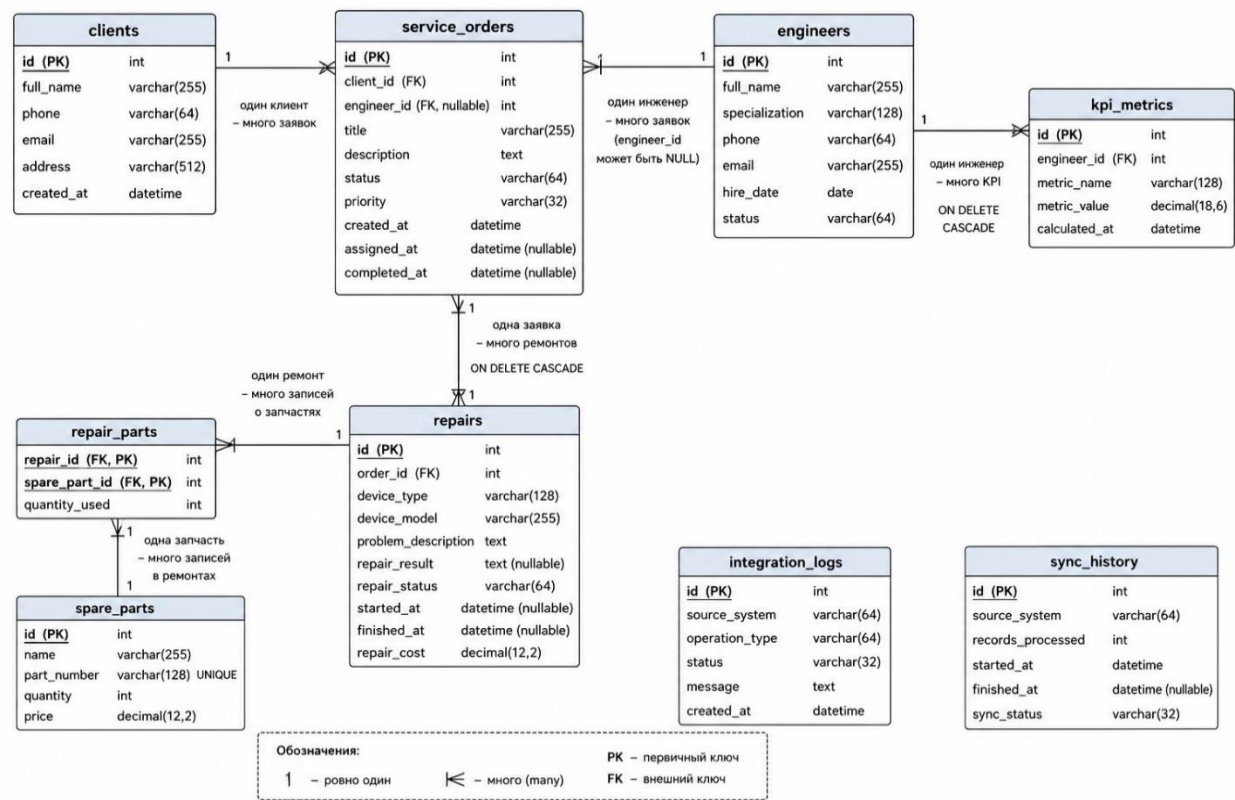


Диаграмма программного комплекса



ER-диаграмма БД



ПРИЛОЖЕНИЕ Б

```
main.py
1 from __future__ import annotations
2 import logging
3 import sys
4 from PyQt6.QtWidgets import QApplication
5 from sqlalchemy.exc import OperationalError
6 from config.settings import get_settings
7 from database.connection import create_engine_from_settings, dispose_engine
8 from database.schema import create_all_tables
9 from database.session import configure_session_factory
10 from ui.main_window import MainWindow, apply_theme
11 from utils.logger import get_logger, setup_logging
12
13 logger = get_logger(__name__)
14
15 def bootstrap_infrastructure() -> None:
16     """Подключение к MySQL, создание таблиц при необходимости."""
17     settings = get_settings()
18     setup_logging(settings.app_log_level)
19     engine = create_engine_from_settings(settings)
20     create_all_tables(engine)
21     configure_session_factory()
22     logger.info("Инфраструктура БД готова: %s", settings.db_name)
23
24 def main() -> int:
25     try:
26         bootstrap_infrastructure()
27     except OperationalError as exc:
28         logging.basicConfig(level=logging.ERROR)
29         logging.exception("Не удалось инициализировать базу данных: %s", exc)
30         if exc.orig and getattr(exc.orig, "args", None) and exc.orig.args[0] == 1045:
31             logging.error(
32                 "Отказ MySQL в доступе: проверьте DB_USER и DB_PASSWORD в `.env` "
33                 "(пароль не должен оставаться примером из шаблона).")
34         )
35         return 1
36     except Exception as exc:
37         logging.basicConfig(level=logging.ERROR)
38         logging.exception("Не удалось инициализировать базу данных: %s", exc)
39         return 1
40
41 app = QApplication(sys.argv)
42 apply_theme(app)
43 window = MainWindow()
44
45 def handle_exception(exc_type, exc, tb) -> None:
46     logger.error("Необработанное исключение UI", exc_info=(exc_type, exc, tb))
47
48 sys.excepthook = handle_exception
49 window.show()
50 code = app.exec()
51 dispose_engine()
52 return int(code)
53
54 if __name__ == "__main__":
55     sys.exit(main())
```


test_sla_calculator.py

```
1 from datetime import datetime, timedelta, timezone
2
3 from analytics.sla_calculator import OrderSLAInput, SLACalculator
4
5
6 def test_sla_met_when_completed_within_target():
7     calc = SLACalculator()
8     created = datetime(2025, 1, 1, 8, 0, tzinfo=timezone.utc)
9     completed = created + timedelta(hours=10)
10    row = OrderSLAInput(
11        order_id=1,
12        priority="high",
13        status="completed",
14        created_at=created,
15        completed_at=completed,
16    )
17    res = calc.evaluate_order(row, now=completed)
18    assert res.breached is False
19
20
21 def test_sla_breach_when_overdue_open_ticket():
22     calc = SLACalculator()
23     created = datetime(2025, 1, 1, 8, 0, tzinfo=timezone.utc)
24     row = OrderSLAInput(
25         order_id=2,
26         priority="high",
27         status="in_progress",
28         created_at=created,
29         completed_at=None,
30     )
31     now = created + timedelta(hours=48)
32     res = calc.evaluate_order(row, now=now)
33     assert res.breached is True
34
```

bitrix_mapper.py

```
1 from __future__ import annotations
2
3 from typing import Any, Dict, List
4
5
6 def deals_to_internal(deals: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
7     """Pass-through with shallow copy – ETL transformer owns semantics."""
8     return [dict(d) for d in deals]
9
```

```

1 from __future__ import annotations
2 import json
3 import os
4 from typing import Any, List
5 import requests
6 from utils.logger import get_logger
7
8 logger = get_logger(__name__)
9
10 MOCK_DEALS_JSON = """
11 [
12     {
13         "ID": "BX-1001",
14         "TITLE": "He включается ноутбук Dell",
15         "COMMENTS": "Клиент просит срочный выезд",
16         "PRIORITY": "high",
17         "STAGE_ID": "assigned",
18         "CONTACT": {"NAME": "Иван Петров", "PHONE": "+79990001122", "EMAIL": "ivan.p@corp.demo"}
19     },
20     {
21         "ID": "BX-1002",
22         "TITLE": "Замена матрицы моноблока",
23         "COMMENTS": "",
24         "PRIORITY": "medium",
25         "STAGE_ID": "in progress",
26         "CONTACT": {"NAME": "ООО Технолайн", "PHONE": "+74951234567", "EMAIL": "svc@technoline.demo"}
27     },
28     {
29         "ID": "BX-1003",
30         "TITLE": "Диагностика ИБП",
31         "COMMENTS": "Объект: склад №4",
32         "PRIORITY": "low",
33         "STAGE_ID": "new",
34         "CONTACT": {"NAME": "Светлана Орлова", "PHONE": "+79887776655", "EMAIL": "orlova@mail.demo"}
35     }
36 ]"""
37
38 class BitrixMockClient:
39     """Имитация CRM.deal.list – возвращает структурированный JSON."""
40
41     def __init__(self, webhook_url: str | None = None) -> None:
42         self._webhook = webhook_url or os.getenv("BITRIX_WEBHOOK_URL")
43
44     def fetch_service_orders(self) -> List[dict[str, Any]]:
45         if self._webhook:
46             try:
47                 url = self._webhook.rstrip("/") + "/crm.deal.list"
48                 resp = requests.get(url, timeout=15)
49                 resp.raise_for_status()
50                 payload = resp.json()
51                 return payload.get("result", payload)
52             except Exception as exc:
53                 logger.error("Bitrix REST failure, fallback to mock: %s", exc)
54         return json.loads(MOCK_DEALS_JSON)
55

```

```

bitrix_sync.py • Конструктор Макет Ссылки Рассылки Рецензирование Вид С
1 > from __future__ import annotations...
10
11 logger = get_logger(__name__)
12
13 class BitrixSyncService:
14     SOURCE = "bitrix24"
15     def __init__(self, session: Session) -> None:
16         self.session = session
17         self._history = SyncHistoryRepository(session)
18         self._logs = IntegrationLogRepository(session)
19
20     def run(self) -> int:
21         started = datetime.now(timezone.utc)
22         hist = SyncHistory(
23             source_system=self.SOURCE,
24             records_processed=0,
25             started_at=started,
26             finished_at=None,
27             sync_status="running",
28         )
29         self._history.add(hist)
30         self.session.flush()
31         try:
32             pipeline = ETLPipeline(self._session)
33             result = pipeline.run_bitrix_orders()
34             hist.records_processed = result.rows_loaded
35             hist.sync_status = "success"
36             hist.finished_at = datetime.now(timezone.utc)
37             self._history.update(hist)
38             self._logs.add(
39                 IntegrationLog(
40                     source_system=self.SOURCE,
41                     operation_type="sync_orders",
42                     status="success",
43                     message=f"processed_in={result.rows_in}, loaded={result.rows_loaded}",
44                 )
45             )
46             logger.info("Bitrix sync finished: %s", result)
47             return result.rows_loaded
48         except Exception as exc:
49             logger.error("Bitrix sync failed: %s", exc)
50             hist.sync_status = "failed"
51             hist.finished_at = datetime.now(timezone.utc)
52             self._history.update(hist)
53             self._logs.add(
54                 IntegrationLog(
55                     source_system=self.SOURCE,
56                     operation_type="sync_orders",
57                     status="failed",
58                     message=str(exc),
59                 )
60             )
61             raise
62
63

```


excel_exporter.py

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from sqlalchemy.orm import Session
6
7 from services.reporting_service import ReportingService
8
9
10 class ExcelAnalyticsExporter:
11     def __init__(self, session: Session) -> None:
12         self._reporting = ReportingService(session)
13
14     def export(self, output_path: str | Path) -> Path:
15         return self._reporting.export_excel(output_path)
16
```

excel_importer.py

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from sqlalchemy.orm import Session
6
7 from etl.pipeline import ETLPipeline
8
9
10 class ExcelImportService:
11     def __init__(self, session: Session) -> None:
12         self._session = session
13
14     def import_orders_workbook(self, path: str | Path) -> int:
15         pipeline = ETLPipeline(self._session)
16         result = pipeline.run_excel_orders(path)
17         return result.rows_loaded
18
```



```
onec_mapper.py • Конструктор Макет Ссылки Рассылки Рецензирование
1 from __future__ import annotations
2
3 from typing import Any, Dict, List
4
5
6 def bundle_to_repairs(rows: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
7     """Ensures transformer keys exist (`repair_status`, `device_type`, ...)."""
8     out: List[Dict[str, Any]] = []
9     for row in rows:
10         row = dict(row)
11         row.setdefault("repair_status", row.get("STATUS"))
12         row.setdefault("device_type", row.get("DEVICE"))
13         row.setdefault("device_model", row.get("MODEL"))
14         row.setdefault("problem_description", row.get("PROBLEM"))
15         row.setdefault("started_at", row.get("STARTED_AT"))
16         row.setdefault("finished_at", row.get("FINISHED_AT"))
17         row.setdefault("repair_cost", row.get("COST"))
18         row.setdefault("order_ref", row.get("ORDER_REF"))
19         out.append(row)
20     return out
21
```

```

1 > from __future__ import annotations...
13
14 logger = get_logger(__name__)
15
16 class OneCSyncService:
17     SOURCE = "onec"
18     def __init__(self, session: Session) -> None:
19         self._session = session
20         self._history = SyncHistoryRepository(session)
21         self._logs = IntegrationLogRepository(session)
22
23     def run(self) -> int:
24         started = datetime.now(timezone.utc)
25         hist = SyncHistory(
26             source_system=self.SOURCE,
27             records_processed=0,
28             started_at=started,
29             finished_at=None,
30             sync_status="running",
31         )
32         self._history.add(hist)
33         self._session.flush()
34         try:
35             batch = OneCExtractor().extract()
36             mapped = bundle_to_repairs(batch.records)
37             transformer = ETLTransformer()
38             norm = transformer.transform_repairs(mapped)
39             loader = ETLLoader(self._session)
40             loaded = loader.load_repairs(norm)
41             hist.records_processed = loaded
42             hist.sync_status = "success"
43             hist.finished_at = datetime.now(timezone.utc)
44             self._history.update(hist)
45             self._logs.add(
46                 IntegrationLog(
47                     source_system=self.SOURCE,
48                     operation_type="sync_repairs",
49                     status="success",
50                     message=f"processed_in={len(batch.records)}, loaded={loaded}",
51                 )
52             )
53             logger.info("1C sync finished, loaded=%s", loaded)
54             return loaded
55         except Exception as exc:
56             logger.error("1C sync failed: %s", exc)
57             hist.sync_status = "failed"
58             hist.finished_at = datetime.now(timezone.utc)
59             self._history.update(hist)
60             self._logs.add(
61                 IntegrationLog(
62                     source_system=self.SOURCE,
63                     operation_type="sync_repairs",
64                     status="failed",
65                     message=str(exc),
66                 )

```

```

main_window.py
1 from __future__ import annotations
2 from PyQt6.QtCore import QSize
3 from PyQt6.QtWidgets import (
4     QFrame,
5     QHBoxLayout,
6     QLabel,
7     QMainWindow,
8     QMessageBox,
9     QPushButton,
10    QStackedWidget,
11    QVBoxLayout,
12    QWidget,
13 )
14 from ui.pages.analytics_page import AnalyticsPage
15 from ui.pages.dashboard_page import DashboardPage
16 from ui.pages.integrations_page import IntegrationsPage
17 from ui.pages.orders_page import OrdersPage
18 from ui.pages.repairs_page import RepairsPage
19 from ui.pages.reports_page import ReportsPage
20 from ui.styles import APP_STYLESHEET
21
22 class MainWindow(QMainWindow):
23     def __init__(self, parent=None) -> None:
24         super().__init__(parent)
25         self.setWindowTitle("Корпоративная аналитика выездного сервиса")
26         self.resize(QSize(1280, 820))
27
28         container = QWidget()
29         layout = QHBoxLayout(container)
30         layout.setContentsMargins(0, 0, 0, 0)
31         layout.setSpacing(0)
32
33         sidebar = QFrame()
34         sidebar.setObjectName("sidebar")
35         sidebar.setFixedWidth(240)
36         side_layout = QVBoxLayout(sidebar)
37         brand = QLabel("SERVICE\nANALYTICS")
38         brand.setStyleSheet("color:#f9fafb;font-weight:700;font-size:16px;padding:16px;")
39         side_layout.addWidget(brand)
40
41         self._stack = QStackedWidget()
42         self._stack.setObjectName("contentStack")
43         self._pages = {
44             "dashboard": DashboardPage(),
45             "orders": OrdersPage(),
46             "repairs": RepairsPage(),
47             "analytics": AnalyticsPage(),
48             "integrations": IntegrationsPage(),
49             "reports": ReportsPage(),
50         }
51
52         self._buttons: dict[str, QPushButton] = {}

```

```

main_window.py • Конструктор Макет Ссылки Рассылки Рецензирование Вид C
22 class MainWindow(QMainWindow):
23     def __init__(self, parent=None) -> None:
54         def add_nav(key: str, title: str) -> None:
56             btn.setCheckable(True)
57             btn.setProperty("class", "nav")
58             btn.clicked.connect(lambda checked=False, k=key: self._open_page(k))
59             side_layout.addWidget(btn)
60             self._buttons[key] = btn
61             self._stack.addWidget(self._pages[key])
62
63         add_nav("dashboard", "Дашборд")
64         add_nav("orders", "Заявки")
65         add_nav("repairs", "Ремонты")
66         add_nav("analytics", "Аналитика")
67         add_nav("integrations", "Интеграции")
68         add_nav("reports", "Отчёты")
69
70         side_layout.addStretch(1)
71
72         layout.addWidget(sidebar)
73         layout.addWidget(self._stack, stretch=1)
74
75         self.setCentralWidget(container)
76
77         self._apply_exclusive_nav("dashboard")
78         try:
79             self._pages["dashboard"].refresh()
80         except Exception as exc:
81             QMessageBox.critical(self, "Ошибка", f"Не удалось загрузить дашборд:\n{exc}")
82
83     def _apply_exclusive_nav(self, active_key: str) -> None:
84         for key, btn in self._buttons.items():
85             btn.setChecked(key == active_key)
86
87     def _open_page(self, key: str) -> None:
88         try:
89             self._apply_exclusive_nav(key)
90             mapping = {
91                 "dashboard": 0,
92                 "orders": 1,
93                 "repairs": 2,
94                 "analytics": 3,
95                 "integrations": 4,
96                 "reports": 5,
97             }
98             self._stack.setCurrentIndex(mapping[key])
99             page = self._pages[key]
100             if hasattr(page, "refresh"):
101                 page.refresh()
102         except Exception as exc:
103             QMessageBox.critical(self, "Ошибка", str(exc))
104
105     def apply_theme(app) -> None:
106         app.setStyle("Fusion")
107         app.setStyleSheet(APP_STYLESHEET)

```


kpi_repository.py

КонструкторМакетСсылкиРассылкиРецензированиеВид

```
1 from __future__ import annotations
2
3 from datetime import datetime
4 from typing import List, Optional
5
6 from sqlalchemy import select
7 from sqlalchemy.orm import Session
8
9 from models.kpi_metric import KPIMetric
10
11
12 class KPIRepository:
13     def __init__(self, session: Session) -> None:
14         self._session = session
15
16     def add_metric(self, metric: KPIMetric) -> KPIMetric:
17         self._session.add(metric)
18         self._session.flush()
19         return metric
20
21     def add_many(self, metrics: List[KPIMetric]) -> None:
22         self._session.add_all(metrics)
23         self._session.flush()
24
25     def latest_for_engineer(
26         self, engineer_id: int, metric_name: str
27     ) -> Optional[KPIMetric]:
28         stmt = (
29             select(KPIMetric)
30             .where(
31                 KPIMetric.engineer_id == engineer_id,
32                 KPIMetric.metric_name == metric_name,
33             )
34             .order_by(KPIMetric.calculated_at.desc())
35             .limit(1)
36         )
37         return self._session.scalars(stmt).first()
38
39     def list_recent(self, limit: int = 500) -> List[KPIMetric]:
40         stmt = select(KPIMetric).order_by(KPIMetric.calculated_at.desc()).limit(limit)
41         return list(self._session.scalars(stmt))
42
43     def list_since(self, since: datetime, limit: int = 5000) -> List[KPIMetric]:
44         stmt = (
45             select(KPIMetric)
46             .where(KPIMetric.calculated_at >= since)
47             .order_by(KPIMetric.calculated_at.desc())
48             .limit(limit)
49         )
50         return list(self._session.scalars(stmt))
51
```